

# BandHiC

Weibing Wang

Feb 08, 2026



# CONTENTS

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Quick Start</b>                                   | <b>11</b> |
| 1.1      | Overview                                             | 11        |
| 1.2      | Installation                                         | 11        |
| 1.2.1    | Required Package                                     | 11        |
| 1.2.2    | Option 1: Install via pip                            | 12        |
| 1.2.3    | Option 2: Install from source with conda             | 12        |
| 1.2.4    | Optional dependency for .hic file support: hic-straw | 12        |
| 1.2.4.1  | Method 1: Install via pip                            | 12        |
| 1.2.4.2  | Method 2: Install via Conda                          | 13        |
| 1.2.4.3  | Upstream installation guide                          | 13        |
| 1.3      | What is BandHiC?                                     | 13        |
| 1.3.1    | Data structure of BandHiC                            | 13        |
| 1.3.2    | Functions of BandHiC                                 | 14        |
| <b>2</b> | <b>Tutorials</b>                                     | <b>15</b> |
| 2.1      | Prerequisites                                        | 15        |
| 2.2      | Import bandhic package                               | 15        |
| 2.3      | Initialize a band_hic_matrix object                  | 16        |
| 2.4      | Load or save a band_hic_matrix object                | 16        |
| 2.5      | Construct a band_hic_matrix object                   | 16        |
| 2.6      | Indexing on band_hic_matrix                          | 17        |
| 2.7      | Masking                                              | 18        |
| 2.8      | Universal functions (ufunc)                          | 19        |
| 2.9      | Other Array Functions                                | 20        |
| <b>3</b> | <b>API Reference</b>                                 | <b>23</b> |
| 3.1      | band_hic_matrix Class                                | 23        |
| 3.1.1    | Class Overview and Initialization                    | 24        |
| 3.1.2    | Masking Methods                                      | 27        |
| 3.1.2.1  | bandhic.band_hic_matrix.init_mask                    | 27        |
| 3.1.2.2  | bandhic.band_hic_matrix.add_mask                     | 28        |
| 3.1.2.3  | bandhic.band_hic_matrix.get_mask                     | 28        |
| 3.1.2.4  | bandhic.band_hic_matrix.unmask                       | 29        |
| 3.1.2.5  | bandhic.band_hic_matrix.drop_mask                    | 29        |
| 3.1.2.6  | bandhic.band_hic_matrix.add_mask_row_col             | 30        |
| 3.1.2.7  | bandhic.band_hic_matrix.get_mask_row_col             | 31        |
| 3.1.2.8  | bandhic.band_hic_matrix.drop_mask_row_col            | 31        |
| 3.1.2.9  | bandhic.band_hic_matrix.clear_mask                   | 31        |
| 3.1.2.10 | bandhic.band_hic_matrix.count_masked                 | 32        |
| 3.1.2.11 | bandhic.band_hic_matrix.count_unmasked               | 32        |

|          |                                               |    |
|----------|-----------------------------------------------|----|
| 3.1.2.12 | bandhic.band_hic_matrix.count_in_band_masked  | 32 |
| 3.1.2.13 | bandhic.band_hic_matrix.count_out_band_masked | 33 |
| 3.1.2.14 | bandhic.band_hic_matrix.count_in_band         | 33 |
| 3.1.2.15 | bandhic.band_hic_matrix.count_out_band        | 34 |
| 3.1.3    | Data Indexing and Modification                | 34 |
| 3.1.3.1  | bandhic.band_hic_matrix.__getitem__           | 35 |
| 3.1.3.2  | bandhic.band_hic_matrix.__setitem__           | 37 |
| 3.1.3.3  | bandhic.band_hic_matrix.get_values            | 39 |
| 3.1.3.4  | bandhic.band_hic_matrix.set_values            | 39 |
| 3.1.3.5  | bandhic.band_hic_matrix.diag                  | 40 |
| 3.1.3.6  | bandhic.band_hic_matrix.set_diag              | 41 |
| 3.1.3.7  | bandhic.band_hic_matrix.extract_row           | 41 |
| 3.1.3.8  | bandhic.band_hic_matrix.__iter__              | 42 |
| 3.1.3.9  | bandhic.band_hic_matrix.iterwindows           | 42 |
| 3.1.3.10 | bandhic.band_hic_matrix.iterrows              | 43 |
| 3.1.3.11 | bandhic.band_hic_matrix.itercols              | 43 |
| 3.1.3.12 | bandhic.band_hic_matrix.filled                | 44 |
| 3.1.3.13 | bandhic.band_hic_matrix.clip                  | 45 |
| 3.1.4    | Data Reduction and Normalization              | 45 |
| 3.1.4.1  | bandhic.band_hic_matrix.min                   | 46 |
| 3.1.4.2  | bandhic.band_hic_matrix.max                   | 47 |
| 3.1.4.3  | bandhic.band_hic_matrix.sum                   | 48 |
| 3.1.4.4  | bandhic.band_hic_matrix.mean                  | 48 |
| 3.1.4.5  | bandhic.band_hic_matrix.std                   | 49 |
| 3.1.4.6  | bandhic.band_hic_matrix.var                   | 50 |
| 3.1.4.7  | bandhic.band_hic_matrix.prod                  | 50 |
| 3.1.4.8  | bandhic.band_hic_matrix.ptp                   | 51 |
| 3.1.4.9  | bandhic.band_hic_matrix.all                   | 52 |
| 3.1.4.10 | bandhic.band_hic_matrix.any                   | 52 |
| 3.1.4.11 | bandhic.band_hic_matrix.normalize             | 53 |
| 3.1.5    | Vectorized Computation Methods                | 53 |
| 3.1.5.1  | bandhic.band_hic_matrix.absolute              | 55 |
| 3.1.5.2  | bandhic.band_hic_matrix.add                   | 56 |
| 3.1.5.3  | bandhic.band_hic_matrix.arccos                | 56 |
| 3.1.5.4  | bandhic.band_hic_matrix.arccosh               | 57 |
| 3.1.5.5  | bandhic.band_hic_matrix.arcsin                | 57 |
| 3.1.5.6  | bandhic.band_hic_matrix.arcsinh               | 58 |
| 3.1.5.7  | bandhic.band_hic_matrix.arctan                | 58 |
| 3.1.5.8  | bandhic.band_hic_matrix.arctan2               | 59 |
| 3.1.5.9  | bandhic.band_hic_matrix.arctanh               | 60 |
| 3.1.5.10 | bandhic.band_hic_matrix.bitwise_and           | 60 |
| 3.1.5.11 | bandhic.band_hic_matrix.bitwise_or            | 61 |
| 3.1.5.12 | bandhic.band_hic_matrix.bitwise_xor           | 61 |
| 3.1.5.13 | bandhic.band_hic_matrix.cbRT                  | 62 |
| 3.1.5.14 | bandhic.band_hic_matrix.conj                  | 62 |
| 3.1.5.15 | bandhic.band_hic_matrix.conjugate             | 63 |
| 3.1.5.16 | bandhic.band_hic_matrix.cos                   | 63 |
| 3.1.5.17 | bandhic.band_hic_matrix.cosh                  | 64 |
| 3.1.5.18 | bandhic.band_hic_matrix.deg2rad               | 64 |
| 3.1.5.19 | bandhic.band_hic_matrix.degrees               | 65 |
| 3.1.5.20 | bandhic.band_hic_matrix.divide                | 65 |
| 3.1.5.21 | bandhic.band_hic_matrix.divmod                | 66 |
| 3.1.5.22 | bandhic.band_hic_matrix.equal                 | 67 |
| 3.1.5.23 | bandhic.band_hic_matrix.exp                   | 67 |

|          |                                                   |    |
|----------|---------------------------------------------------|----|
| 3.1.5.24 | bandhic.band_hic_matrix.exp2 . . . . .            | 68 |
| 3.1.5.25 | bandhic.band_hic_matrix.expm1 . . . . .           | 68 |
| 3.1.5.26 | bandhic.band_hic_matrix.fabs . . . . .            | 69 |
| 3.1.5.27 | bandhic.band_hic_matrix.float_power . . . . .     | 69 |
| 3.1.5.28 | bandhic.band_hic_matrix.floor_divide . . . . .    | 70 |
| 3.1.5.29 | bandhic.band_hic_matrix.fmod . . . . .            | 70 |
| 3.1.5.30 | bandhic.band_hic_matrix.gcd . . . . .             | 71 |
| 3.1.5.31 | bandhic.band_hic_matrix.greater . . . . .         | 72 |
| 3.1.5.32 | bandhic.band_hic_matrix.greater_equal . . . . .   | 72 |
| 3.1.5.33 | bandhic.band_hic_matrix.heaviside . . . . .       | 73 |
| 3.1.5.34 | bandhic.band_hic_matrix.hypot . . . . .           | 73 |
| 3.1.5.35 | bandhic.band_hic_matrix.invert . . . . .          | 74 |
| 3.1.5.36 | bandhic.band_hic_matrix.lcm . . . . .             | 74 |
| 3.1.5.37 | bandhic.band_hic_matrix.left_shift . . . . .      | 75 |
| 3.1.5.38 | bandhic.band_hic_matrix.less . . . . .            | 76 |
| 3.1.5.39 | bandhic.band_hic_matrix.less_equal . . . . .      | 76 |
| 3.1.5.40 | bandhic.band_hic_matrix.log . . . . .             | 77 |
| 3.1.5.41 | bandhic.band_hic_matrix.log1p . . . . .           | 77 |
| 3.1.5.42 | bandhic.band_hic_matrix.log2 . . . . .            | 78 |
| 3.1.5.43 | bandhic.band_hic_matrix.log10 . . . . .           | 78 |
| 3.1.5.44 | bandhic.band_hic_matrix.logaddexp . . . . .       | 79 |
| 3.1.5.45 | bandhic.band_hic_matrix.logaddexp2 . . . . .      | 79 |
| 3.1.5.46 | bandhic.band_hic_matrix.logical_and . . . . .     | 80 |
| 3.1.5.47 | bandhic.band_hic_matrix.logical_or . . . . .      | 80 |
| 3.1.5.48 | bandhic.band_hic_matrix.logical_xor . . . . .     | 81 |
| 3.1.5.49 | bandhic.band_hic_matrix.maximum . . . . .         | 82 |
| 3.1.5.50 | bandhic.band_hic_matrix.minimum . . . . .         | 82 |
| 3.1.5.51 | bandhic.band_hic_matrix.mod . . . . .             | 83 |
| 3.1.5.52 | bandhic.band_hic_matrix.multiply . . . . .        | 83 |
| 3.1.5.53 | bandhic.band_hic_matrix.negative . . . . .        | 84 |
| 3.1.5.54 | bandhic.band_hic_matrix.not_equal . . . . .       | 84 |
| 3.1.5.55 | bandhic.band_hic_matrix.positive . . . . .        | 85 |
| 3.1.5.56 | bandhic.band_hic_matrix.power . . . . .           | 86 |
| 3.1.5.57 | bandhic.band_hic_matrix.rad2deg . . . . .         | 86 |
| 3.1.5.58 | bandhic.band_hic_matrix.radians . . . . .         | 87 |
| 3.1.5.59 | bandhic.band_hic_matrix.reciprocal . . . . .      | 87 |
| 3.1.5.60 | bandhic.band_hic_matrix.remainder . . . . .       | 88 |
| 3.1.5.61 | bandhic.band_hic_matrix.right_shift . . . . .     | 88 |
| 3.1.5.62 | bandhic.band_hic_matrix rint . . . . .            | 89 |
| 3.1.5.63 | bandhic.band_hic_matrix.sign . . . . .            | 89 |
| 3.1.5.64 | bandhic.band_hic_matrix.sin . . . . .             | 90 |
| 3.1.5.65 | bandhic.band_hic_matrix.sinh . . . . .            | 90 |
| 3.1.5.66 | bandhic.band_hic_matrix.sqrt . . . . .            | 91 |
| 3.1.5.67 | bandhic.band_hic_matrix.square . . . . .          | 91 |
| 3.1.5.68 | bandhic.band_hic_matrix.subtract . . . . .        | 92 |
| 3.1.5.69 | bandhic.band_hic_matrix.tan . . . . .             | 92 |
| 3.1.5.70 | bandhic.band_hic_matrix.tanh . . . . .            | 93 |
| 3.1.5.71 | bandhic.band_hic_matrix.true_divide . . . . .     | 93 |
| 3.1.6    | Data Type Conversion and Representation . . . . . | 94 |
| 3.1.6.1  | bandhic.band_hic_matrix.todense . . . . .         | 94 |
| 3.1.6.2  | bandhic.band_hic_matrix.tocoo . . . . .           | 95 |
| 3.1.6.3  | bandhic.band_hic_matrix.tocsr . . . . .           | 95 |
| 3.1.6.4  | bandhic.band_hic_matrix.copy . . . . .            | 96 |
| 3.1.6.5  | bandhic.band_hic_matrix.astype . . . . .          | 96 |

|         |                                            |     |
|---------|--------------------------------------------|-----|
| 3.1.6.6 | bandhic.band_hic_matrix.__repr__           | 97  |
| 3.1.6.7 | bandhic.band_hic_matrix.__str__            | 97  |
| 3.1.6.8 | bandhic.band_hic_matrix.__array__          | 97  |
| 3.1.6.9 | bandhic.band_hic_matrix.__array_priority__ | 98  |
| 3.1.7   | Other Methods                              | 98  |
| 3.1.7.1 | bandhic.band_hic_matrix.memory_usage       | 98  |
| 3.1.7.2 | bandhic.band_hic_matrix.__len__            | 99  |
| 3.1.7.3 | bandhic.band_hic_matrix.__hash__           | 99  |
| 3.1.7.4 | bandhic.band_hic_matrix.__bool__           | 100 |
| 3.1.7.5 | bandhic.band_hic_matrix.dump               | 100 |
| 3.2     | Creation Functions                         | 100 |
| 3.2.1   | bandhic.ones                               | 101 |
| 3.2.2   | bandhic.zeros                              | 101 |
| 3.2.3   | bandhic.eye                                | 102 |
| 3.2.4   | bandhic.full                               | 102 |
| 3.2.5   | bandhic.ones_like                          | 103 |
| 3.2.6   | bandhic.zeros_like                         | 103 |
| 3.2.7   | bandhic.eye_like                           | 104 |
| 3.2.8   | bandhic.full_like                          | 104 |
| 3.3     | Input/Output Functions                     | 105 |
| 3.3.1   | bandhic.save_npz                           | 105 |
| 3.3.2   | bandhic.load_npz                           | 106 |
| 3.3.3   | bandhic.straw_chr                          | 106 |
| 3.3.4   | bandhic.straw_all_chrs                     | 107 |
| 3.3.5   | bandhic.cooler_chr                         | 108 |
| 3.3.6   | bandhic.cooler_all_chrs                    | 109 |
| 3.4     | Vectorized operations for BandHiC matrices | 109 |
| 3.4.1   | bandhic.absolute                           | 111 |
| 3.4.2   | bandhic.add                                | 112 |
| 3.4.3   | bandhic.arccos                             | 112 |
| 3.4.4   | bandhic.arccosh                            | 113 |
| 3.4.5   | bandhic.arcsin                             | 113 |
| 3.4.6   | bandhic.arcsinh                            | 114 |
| 3.4.7   | bandhic.arctan                             | 115 |
| 3.4.8   | bandhic.arctan2                            | 115 |
| 3.4.9   | bandhic.arctanh                            | 116 |
| 3.4.10  | bandhic.bitwise_and                        | 116 |
| 3.4.11  | bandhic.bitwise_or                         | 117 |
| 3.4.12  | bandhic.bitwise_xor                        | 117 |
| 3.4.13  | bandhic.cbrt                               | 118 |
| 3.4.14  | bandhic.conj                               | 119 |
| 3.4.15  | bandhic.conjugate                          | 119 |
| 3.4.16  | bandhic.cos                                | 120 |
| 3.4.17  | bandhic.cosh                               | 120 |
| 3.4.18  | bandhic.deg2rad                            | 121 |
| 3.4.19  | bandhic.degrees                            | 121 |
| 3.4.20  | bandhic.divide                             | 122 |
| 3.4.21  | bandhic.divmod                             | 122 |
| 3.4.22  | bandhic.equal                              | 123 |
| 3.4.23  | bandhic.exp                                | 124 |
| 3.4.24  | bandhic.exp2                               | 124 |
| 3.4.25  | bandhic.expm1                              | 125 |
| 3.4.26  | bandhic.fabs                               | 125 |
| 3.4.27  | bandhic.float_power                        | 126 |

|        |                                  |     |
|--------|----------------------------------|-----|
| 3.4.28 | bandhic.floor_divide             | 126 |
| 3.4.29 | bandhic.fmod                     | 127 |
| 3.4.30 | bandhic.gcd                      | 128 |
| 3.4.31 | bandhic.greater                  | 128 |
| 3.4.32 | bandhic.greater_equal            | 129 |
| 3.4.33 | bandhic.heaviside                | 129 |
| 3.4.34 | bandhic.hypot                    | 130 |
| 3.4.35 | bandhic.invert                   | 131 |
| 3.4.36 | bandhic.lcm                      | 131 |
| 3.4.37 | bandhic.left_shift               | 132 |
| 3.4.38 | bandhic.less                     | 132 |
| 3.4.39 | bandhic.less_equal               | 133 |
| 3.4.40 | bandhic.log                      | 134 |
| 3.4.41 | bandhic.log1p                    | 134 |
| 3.4.42 | bandhic.log2                     | 135 |
| 3.4.43 | bandhic.log10                    | 135 |
| 3.4.44 | bandhic.logaddexp                | 136 |
| 3.4.45 | bandhic.logaddexp2               | 136 |
| 3.4.46 | bandhic.logical_and              | 137 |
| 3.4.47 | bandhic.logical_or               | 138 |
| 3.4.48 | bandhic.logical_xor              | 138 |
| 3.4.49 | bandhic.maximum                  | 139 |
| 3.4.50 | bandhic.minimum                  | 139 |
| 3.4.51 | bandhic.mod                      | 140 |
| 3.4.52 | bandhic.multiply                 | 141 |
| 3.4.53 | bandhic.negative                 | 141 |
| 3.4.54 | bandhic.not_equal                | 142 |
| 3.4.55 | bandhic.positive                 | 142 |
| 3.4.56 | bandhic.power                    | 143 |
| 3.4.57 | bandhic.rad2deg                  | 143 |
| 3.4.58 | bandhic.radians                  | 144 |
| 3.4.59 | bandhic.reciprocal               | 144 |
| 3.4.60 | bandhic.remainder                | 145 |
| 3.4.61 | bandhic.right_shift              | 146 |
| 3.4.62 | bandhic rint                     | 146 |
| 3.4.63 | bandhic.sign                     | 147 |
| 3.4.64 | bandhic.sin                      | 147 |
| 3.4.65 | bandhic.sinh                     | 148 |
| 3.4.66 | bandhic.sqrt                     | 148 |
| 3.4.67 | bandhic.square                   | 149 |
| 3.4.68 | bandhic.subtract                 | 149 |
| 3.4.69 | bandhic.tan                      | 150 |
| 3.4.70 | bandhic.tanh                     | 150 |
| 3.4.71 | bandhic.true_divide              | 151 |
| 3.5    | Data Reduction and Normalization | 151 |
| 3.5.1  | bandhic.min                      | 152 |
| 3.5.2  | bandhic.max                      | 152 |
| 3.5.3  | bandhic.sum                      | 152 |
| 3.5.4  | bandhic.mean                     | 152 |
| 3.5.5  | bandhic.std                      | 153 |
| 3.5.6  | bandhic.var                      | 153 |
| 3.5.7  | bandhic.prod                     | 153 |
| 3.5.8  | bandhic.ptp                      | 153 |
| 3.5.9  | bandhic.all                      | 153 |

|          |                                                |            |
|----------|------------------------------------------------|------------|
| 3.5.10   | bandhic.any . . . . .                          | 153        |
| 3.5.11   | bandhic.clip . . . . .                         | 154        |
| 3.5.12   | bandhic.normalize . . . . .                    | 154        |
| 3.6      | Other Functions . . . . .                      | 154        |
| 3.6.1    | bandhic.matrix_equal . . . . .                 | 154        |
| 3.6.2    | bandhic.assert_band_matrix_equal . . . . .     | 155        |
| 3.6.3    | bandhic.compute_bin_bias . . . . .             | 155        |
| 3.6.4    | bandhic.call_tad . . . . .                     | 156        |
| 3.6.5    | bandhic.topdom . . . . .                       | 156        |
| 3.6.6    | bandhic.apa . . . . .                          | 157        |
| 3.6.7    | bandhic.hiccups . . . . .                      | 158        |
| 3.6.8    | bandhic.load_norm_vector_from_hic . . . . .    | 159        |
| 3.6.9    | bandhic.load_norm_vector_from_cooler . . . . . | 159        |
| 3.6.10   | bandhic.scc . . . . .                          | 160        |
| <b>4</b> | <b>Indices and Tables</b>                      | <b>161</b> |
|          | <b>Index</b>                                   | <b>163</b> |



---

Version: 0.2.0

Welcome to the BandHiC documentation!

This documentation provides a comprehensive guide to the BandHiC library, including its features, installation instructions, and API reference. It is designed to help you efficiently store and manipulate Hi-C contact matrices in a banded form, enabling NumPy-compatible operations, matrix masking, diagonal reduction, file I/O, and domain-level interaction analysis.

If you have any questions, please contact us:

Author: Wang Weibing

Email: [wangweibing@xidian.edu.cn](mailto:wangweibing@xidian.edu.cn)

Useful links: [PDF Download](#) | [Website](#) | [PyPI](#) | [GitHub](#)

---

## QUICK START

This section shows basic usage of the **BandHiC** package, including installation and a brief introduction to its features.

### 1.1 Overview

**BandHiC** is a Python package for efficient storage, manipulation, and analysis of Hi-C matrices using a banded matrix representation.

BandHiC adopts a banded storage scheme that stores only a configurable diagonal bandwidth of the dense Hi-C contact matrices. This design can reduce memory usage by up to 99% compared to dense matrices, while still supporting fast random access and user-friendly indexing operations.

Main features include:

- Efficient storage of Hi-C matrices using a banded representation.
- Fast random access and indexing operations.
- Support for flexible masking of missing values, outliers, and unmappable regions.
- Vectorized operations optimized with NumPy for high performance.
- Scalable for ultra-high-resolution Hi-C data analysis.

### 1.2 Installation

#### 1.2.1 Required Package

**BandHiC** can be installed on Linux-like systems and requires the following dependencies:

1. python  $\geq$  3.8
2. numpy  $\geq$  1.24
3. pandas  $\geq$  2.0
4. scipy  $\geq$  1.10
5. cooler  $\geq$  0.10
6. hic\_straw  $\geq$  1.3
7. joblib  $\geq$  1.2
8. numba  $\geq$  0.58

---

9. matplotlib >= 3.7

There are two recommended ways to install **BandHiC**:

### 1.2.2 Option 1: Install via pip

If you already have Python >= 3.8 installed:

```
pip install bandhic
```

If the installation fails due to dependency issues, please manually install the dependencies and then rerun the above command.

### 1.2.3 Option 2: Install from source with conda

1. Clone the repository

```
git clone https://github.com/xdwbb/BandHiC-Master.git  
cd BandHiC-Master
```

2. Create the environment and activate it

```
conda env create -f environment.yml  
conda activate bandhic
```

3. Install BandHiC

```
pip install .
```

### 1.2.4 Optional dependency for .hic file support: hic-straw

Support for reading .hic format Hi-C data relies on the third-party package **hic-straw**, which is **not installed automatically** with BandHiC.

If you do **not** need to read .hic files, you can ignore this dependency and use BandHiC normally.

If you **do** need .hic support, please install hic-straw manually using one of the following methods.

#### 1.2.4.1 Method 1: Install via pip

```
pip install hic-straw
```

Note that hic-straw includes native C/C++ extensions. Installation via pip may require a compatible compiler toolchain and system libraries (e.g. libcurl development headers).

### 1.2.4.2 Method 2: Install via Conda

```
conda install -c bioconda hic-straw
```

Using Conda provides prebuilt binaries on many platforms and avoids local compilation issues.

### 1.2.4.3 Upstream installation guide

For detailed, system-specific installation instructions, please refer to the official *straw* repository maintained by the Aiden Lab:

<https://github.com/aidenlab/straw>

## 1.3 What is BandHiC?

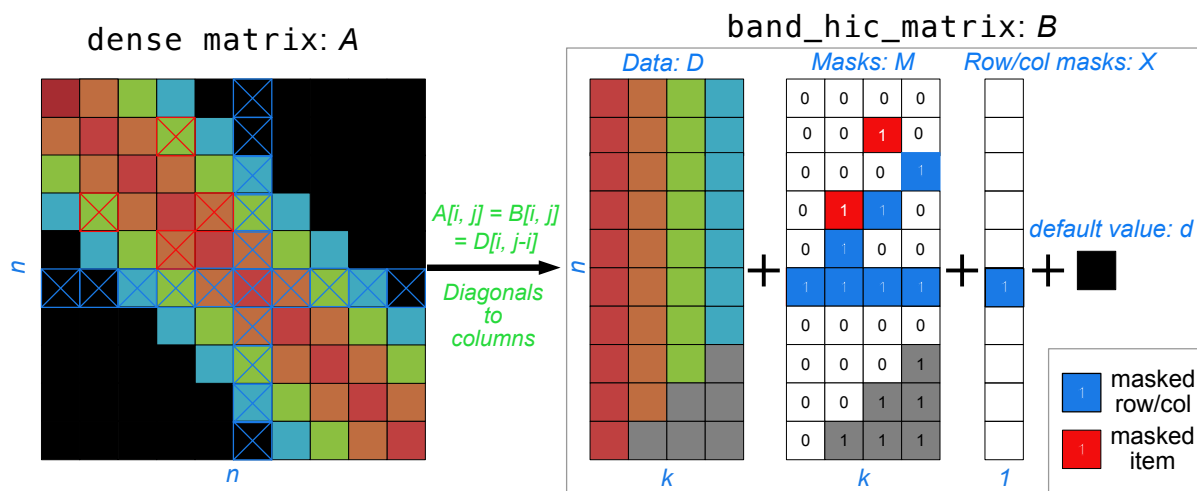


Fig. 1.1: Data structure of BandHiC. Schematic illustration of converting a dense symmetric matrix  $A$  into a banded representation consisting of a data matrix  $D$ , an element-wise mask matrix  $M$ , a row/column mask matrix  $X$ , and a default value  $d$  for out-of-band entries. Diagonal elements from  $A$  are reorganized into columns of  $D$ ;  $M$  marks missing or outlier entries;  $X$  indicates masked rows or columns.

### 1.3.1 Data structure of BandHiC

To address the growing memory demands posed by high-resolution Hi-C data, we introduce `band_hic_matrix`, the core class implemented in the BandHiC package. For a Hi-C contact matrix  $A \in \mathbb{R}^{n \times n}$  at resolution  $r$ , `band_hic_matrix` retains only the diagonals within a user-defined bandwidth  $k$ , yielding a compact representation  $D \in \mathbb{R}^{n \times k}$ . This format ensures that each column in  $D$  corresponds to a fixed diagonal of  $A$ , such that the mapping  $A[i, j] = D[i, j - i]$  holds for  $|i - j| \leq k$ .

The memory efficiency achieved by this strategy is substantial. When  $k \ll n$ , the memory footprint of `band_hic_matrix` is reduced from  $\mathcal{O}(n^2)$  to  $\mathcal{O}(nk)$ . For example, assuming a resolution of 1 kb and a bandwidth of 2 Mb ( $k = 200$ ), the representation of chromosome 1 of the human genome (~249 Mb) requires 3.7 GB of memory, less than 1% of the memory required by the dense matrix (~461.9 GB). This compression enables the use of high-resolution Hi-C data on commodity hardware without sacrificing random access efficiency.

---

To further enhance flexibility of usage, `band_hic_matrix` integrates an optional two-tier masking mechanism. The element-wise mask matrix  $M \in \{0, 1\}^{n \times k}$  allows users to selectively ignore missing or outlier contacts, enabling robust statistical estimation on unmasked subsets. Additionally, a bin-level mask  $X \in \{0, 1\}^n$  supports the exclusion of entire rows or columns, which is particularly useful for removing repetitive genomic regions devoid of a valid Hi-C signal. These masking features facilitate downstream tasks such as the estimation of average contact intensity at given distances, while confirming statistical validity.

Lastly, a scalar default value  $d$  is defined to fill in the undefined entries of  $A$  not covered by the band matrix  $D$ . This default is typically set to 0, consistent with the assumption that long-range interactions are negligibly sparse. It also ensures that reconstruction of the full matrix  $A$  (if required) can be achieved seamlessly by combining  $D$ ,  $M$ ,  $X$ , and  $d$ . Overall, `band_hic_matrix` provides an efficient, flexible data structure for scalable Hi-C data analysis.

### 1.3.2 Functions of BandHiC

A key feature of `band_hic_matrix` is its direct coordinate mapping between the banded matrix  $B$  and the full dense matrix  $A$ . For any pair of genomic loci  $(i, j)$  satisfying the band constraint  $|i - j| \leq k$ , the interaction frequency  $A[i, j]$  can be accessed in constant time via  $D[i, j - i]$ . This structure ensures random access in  $\mathcal{O}(1)$  time, which is critical for performance-sensitive Hi-C analyses, particularly when memory constraints prohibit the use of fully dense matrices. Data access in `band_hic_matrix` is fully consistent with that of a dense matrix, as each entry is accessed via  $B[i, j] = D[i, j - i] = A[i, j]$ , allowing users to interact with `band_hic_matrix` objects as if they were dense matrices without needing to consider the underlying implementation.

Owing to this random-access capability, `band_hic_matrix` supports NumPy-like indexing semantics, including slicing, boolean indexing, and fancy indexing. This design allows users to easily query local chromatin contacts. For instance, a slice operation such as `B[i:j, i:j]` retrieves a banded submatrix. Combined with the `todense` operation, this enables reconstruction of the dense submatrix for downstream analysis or visualization.

In addition to flexible data access, `band_hic_matrix` also supports a wide range of numerical operations, including element-wise arithmetic operations and reduction operations. These operations are implemented using NumPy for efficiency. Standard reduction operations such as `sum`, `min`, and `max` are supported along conventional axes (rows or columns), as well as along the diagonal axis, which is particularly useful in summarizing interaction intensities by genomic distance. This feature facilitates common Hi-C analyses such as distance-decay profiling.

Taken together, `band_hic_matrix` combines the memory efficiency of a banded storage model with the expressiveness of NumPy's interface. By mimicking both NumPy's `ndarray` and `MaskedArray` behaviors, it provides an intuitive and powerful interface for users, substantially lowering the barrier to adoption and promoting integration into existing Hi-C data analysis workflows.

## TUTORIALS

This section provides detailed tutorials on how to use the **BandHiC** package effectively.

- *Prerequisites*
- *Import bandhic package*
- *Initialize a band\_hic\_matrix object*
- *Load or save a band\_hic\_matrix object*
- *Construct a band\_hic\_matrix object*
- *Indexing on band\_hic\_matrix*
- *Masking*
- *Universal functions (ufunc)*
- *Other Array Functions*

### 2.1 Prerequisites

BandHiC can serve as an alternative to the NumPy package when managing and manipulating Hi-C data, aiming to address the issue of excessive memory usage caused by storing dense matrices using NumPy's `ndarray`. At the same time, BandHiC supports masking operations similar to NumPy's `ma.MaskedArray` module, with enhancements tailored for Hi-C data.

Users can leverage their experience with NumPy when using the BandHiC package, so it is recommended that users have some basic knowledge of NumPy. A link to NumPy is provided below: <https://numpy.org>

### 2.2 Import bandhic package

```
>>> import bandhic as bh
```

---

## 2.3 Initialize a band\_hic\_matrix object

Initialize from a SciPy coo\_matrix object:

```
>>> import bandhic as bh
>>> import numpy as np
>>> from scipy.sparse import coo_matrix
>>> coo = coo_matrix(([1, 2, 3], ([0, 1, 2], [0, 1, 2])), shape=(3,3))
>>> mat1 = bh.band_hic_matrix(coo, diag_num=2)
```

Initialize from a tuple (data, (row, col)):

```
>>> mat2 = bh.band_hic_matrix(([4, 5, 6], ([0, 1, 2], [2, 1, 0])), diag_num=1)
```

Initialize from a full dense array, only upper-triangular part is stored, lower part is symmetrized:

```
>>> arr = np.arange(16).reshape(4,4)
>>> mat3 = bh.band_hic_matrix(arr, diag_num=3)
```

## 2.4 Load or save a band\_hic\_matrix object

```
>>> bh.save_npz('./sample.npz', mat)
>>> mat = bh.load_npz('./sample.npz')
```

Load from .hic file:

```
>>> mat = bh.straw_chr('sample.hic',
                      'chr1',
                      resolution=10000,
                      diag_num=200
                      )
```

Load from .mcool file:

```
>>> mat = bh.cooler_chr('sample.mcool',
                       'chr1',
                       diag_num=200
                       resolution=10000,
                       )
```

## 2.5 Construct a band\_hic\_matrix object

Create a band\_hic\_matrix object filled with zeros.

```
>>> mat1 = bh.zeros((5, 5), diag_num=3, dtype=float)
```

Create a band\_hic\_matrix object filled with ones.

```
>>> mat2 = bh.ones((5, 5), diag_num=3, dtype=float)
```



---

Create a `band_hic_matrix` object filled as an identity matrix.

```
>>> mat3 = bh.eye((5, 5), diag_num=3, dtype=float)
```

Create a `band_hic_matrix` object filled with a specified value.

```
>>> mat4 = bh.full((5, 5), fill_value=0.1, diag_num=3, dtype=float)
```

Create a `band_hic_matrix` object matching another matrix, filled with zeros.

```
>>> mat5 = bh.zeros_like(mat1, diag_num=3, dtype=float)
```

Create a `band_hic_matrix` object matching another matrix, filled with ones.

```
>>> mat6 = bh.ones_like(mat1, diag_num=3, dtype=float)
```

Create a `band_hic_matrix` object matching another matrix, filled as an identity matrix.

```
>>> mat7 = bh.eye_like(mat1, diag_num=3, dtype=float)
```

Create a `band_hic_matrix` object matching another matrix, filled with a specified value.

```
>>> mat8 = bh.full_like(mat1, fill_value=0.1, diag_num=3, dtype=float)
```

## 2.6 Indexing on `band_hic_matrix`

First, we create a `band_hic_matrix` object:

```
>>> import numpy as np
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(16).reshape(4,4), diag_num=2)
```

Single-element access (scalar)

```
>>> mat[1, 2]
6
```

Masked element returns masked

```
>>> mat2 = bh.band_hic_matrix(np.eye(4), dtype=int, diag_num=2, mask=([0],[1]))
>>> mat2[0, 1]
masked
```

Square submatrix via two-slice indexing returns `band_hic_matrix`

```
>>> sub = mat[1:3, 1:3]
>>> isinstance(sub, bh.band_hic_matrix)
True
```

Single-axis slice returns `band_hic_matrix` for square region

```
>>> sub2 = mat[0:2] # equivalent to mat[0:2, 0:2]
>>> isinstance(sub2, bh.band_hic_matrix)
True
```

---

Fancy indexing returns ndarray or MaskedArray

```
>>> arr = mat[[0,2,3], [1,2,0]]
>>> isinstance(arr, np.ndarray)
True

>>> mat.add_mask([0,1],[1,2]) # Add mask to some entries
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True
```

Boolean indexing with band\_hic\_matrix

```
>>> mat3 = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=([0,1],[1,2]))
>>> bool_mask = mat3 > 0 # Create a boolean mask
>>> result = mat3[bool_mask] # Use boolean mask for indexing
>>> isinstance(result, np.ma.MaskedArray)
True
>>> result
masked_array(data=[1.0, 1.0, 1.0, 1.0],
             mask=[False, False, False, False],
             fill_value=0.0)
```

## 2.7 Masking

Add item-wise mask:

```
>>> mat.add_mask([0, 1], [1, 2])
```

Add row/column mask:

```
>>> mask = np.array([True, False, False])
>>> mat.add_mask_row_col(mask)
```

Remove mask for specified indices.

```
>>> mat.unmask(( [0], [1] ))
```

Remove all item-wise mask and row/column mask.

```
>>> mat.unmask()
```

Remove all item-wise mask and row/column mask.

```
>>> mat.clear_mask()
```

Drop all item-wise mask but preserve all row/column mask.

```
>>> mat.drop_mask()
```

Drop all row/column mask.

```
>>> mat.drop_mask_row_col()
```

Access masked *band\_hic\_matrix* will obtain *np.ma.MaskedArray* object:

```
>>> mat.add_mask([0, 1], [1, 2])
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True
```

## 2.8 Universal functions (ufunc)

Universal functions that BandHiC supports:

| Function    | Description                    | Function      | Description                     |
|-------------|--------------------------------|---------------|---------------------------------|
| absolute    | Absolute value                 | add           | Element-wise addition           |
| arccos      | Inverse cosine                 | arccosh       | Inverse hyperbolic cosine       |
| arcsin      | Inverse sine                   | arcsinh       | Inverse hyperbolic sine         |
| arctan      | Inverse tangent                | arctan2       | Arctangent of y/x with quadrant |
| arctanh     | Inverse hyperbolic tangent     | bitwise_and   | Element-wise bitwise AND        |
| bitwise_or  | Element-wise bitwise OR        | bitwise_xor   | Element-wise bitwise XOR        |
| cbrt        | Cube root                      | conj          | Complex conjugate               |
| conjugate   | Alias for conj                 | cos           | Cosine function                 |
| cosh        | Hyperbolic cosine              | deg2rad       | Degrees to radians              |
| degrees     | Radians to degrees             | divide        | Element-wise division           |
| divmod      | Quotient and remainder         | equal         | Element-wise equality test      |
| exp         | Exponential                    | exp2          | Base-2 exponential              |
| expm1       | $\exp(x) - 1$                  | fabs          | Absolute value (float)          |
| float_power | Floating-point power           | floor_divide  | Integer division (floor)        |
| fmod        | Modulo operation               | gcd           | Greatest common divisor         |
| greater     | Element-wise greater-than test | greater_equal | Greater-than or equal test      |
| heaviside   | Heaviside step function        | hypot         | Euclidean norm                  |
| invert      | Bitwise inversion              | lcm           | Least common multiple           |
| left_shift  | Bitwise left shift             | less          | Element-wise less-than test     |
| less_equal  | Less-than or equal test        | log           | Natural logarithm               |
| log1p       | $\log(1 + x)$                  | log2          | Base-2 logarithm                |
| log10       | Base-10 logarithm              | logaddexp     | $\log(\exp(x) + \exp(y))$       |
| logaddexp2  | Base-2 version of logaddexp    | logical_and   | Element-wise logical AND        |
| logical_or  | Element-wise logical OR        | logical_xor   | Element-wise logical XOR        |
| maximum     | Element-wise maximum           | minimum       | Element-wise minimum            |
| mod         | Remainder (modulo)             | multiply      | Element-wise multiplication     |
| negative    | Element-wise negation          | not_equal     | Element-wise inequality test    |
| positive    | Returns input unchanged        | power         | Raise to power                  |
| rad2deg     | Radians to degrees             | radians       | Degrees to radians              |
| reciprocal  | Element-wise reciprocal        | remainder     | Modulo remainder                |
| right_shift | Bitwise right shift            | rint          | Round to nearest integer        |
| sign        | Sign of input                  | sin           | Sine function                   |
| sinh        | Hyperbolic sine                | sqrt          | Square root                     |
| square      | Square of input                | subtract      | Element-wise subtraction        |
| tan         | Tangent function               | tanh          | Hyperbolic tangent              |

continues on next page

Table 2.1 – continued from previous page

| Function    | Description                 | Function | Description |
|-------------|-----------------------------|----------|-------------|
| true_divide | Division that returns float |          |             |

BandHiC supports these universal functions, and they can be used in the following four ways:

1. As methods of the `band_hic_matrix` object:

When two `band_hic_matrix` objects are involved, their shape and `diag_num` must match

```
>>> mat3 = mat1.add(mat2)
>>> mat4 = mat1.less(mat2)
>>> mat5 = mat1.negative()
```

2. As functions of the BandHiC package

```
>>> mat3 = bh.add(mat1, mat2)
>>> mat4 = bh.less(mat1, mat2)
>>> mat5 = bh.negative(mat1)
```

3. Using mathematical operators:

```
>>> mat3 = mat1 + mat2
>>> mat4 = mat1 < mat2
>>> mat5 = - mat1
```

4. Calling NumPy's universal functions:

```
>>> mat3 = np.add(mat1, mat2)
>>> mat4 = np.less(mat1, mat2)
>>> mat5 = np.negative(mat1)
```

## 2.9 Other Array Functions

| Function | Description                                                  |
|----------|--------------------------------------------------------------|
| sum      | Compute the sum of all elements along the specified axis     |
| prod     | Compute the product of all elements along the specified axis |
| min      | Return the minimum value along the specified axis            |
| max      | Return the maximum value along the specified axis            |
| mean     | Compute the arithmetic mean along the specified axis         |
| var      | Compute the variance (average squared deviation)             |
| std      | Compute the standard deviation (square root of variance)     |
| ptp      | Compute the range (max - min) of values along the axis       |
| all      | Return True if all elements evaluate to True                 |
| any      | Return True if any element evaluates to True                 |
| clip     | Limit values to a specified min and max range                |

BandHiC supports these functions, and they can be used in the following three ways:

1. As methods of the `band_hic_matrix` object:

Compute the sum of all elements including out-of-band values filled with *default\_value*.

---

```
>>> result0 = mat1.sum()
```

Compute the sum of all elements along the *row* axis

```
>>> result1 = mat1.sum(axis=0)
>>> result1 = mat1.sum(axis='row')
```

Compute the sum of all elements along the *diag* axis

```
>>> result2 = mat1.sum(axis='diag')
```

2. Calling BandHiC's functions:

```
>>> result0 = bh.sum(mat1)
>>> result1 = bh.sum(mat1, axis=0)
>>> result2 = bh.sum(mat1, axis='diag')
```

3. Calling NumPy's functions:

```
>>> result0 = np.sum(mat1)
>>> result1 = np.sum(mat1, axis=0)
>>> result2 = np.sum(mat1, axis='diag')
```



## API REFERENCE

This page provides a full API overview of the BandHiC module, including the core `band_hic_matrix` class and its utilities.

The *bandhic* module defines data structures and utilities for efficiently storing and manipulating Hi-C contact matrices in a banded form. It enables NumPy-compatible operations, matrix masking, diagonal reduction, file I/O, and domain-level interaction analysis.

Core components:

- *band\_hic\_matrix*: A memory-efficient matrix class supporting banded Hi-C data.
- Utility functions: loaders (*cooler\_chr*) and constructors (*ones*, *zeros*, etc.).

### 3.1 `band_hic_matrix` Class

The `band_hic_matrix` class is the core data structure in the BandHiC package, designed for efficient storage and manipulation of Hi-C contact matrices using a banded representation. It supports various operations similar to those in NumPy, while optimizing memory usage and performance.

- *Class Overview and Initialization*
- *Masking Methods*
- *Data Indexing and Modification*
- *Data Reduction and Normalization*
- *Vectorized Computation Methods*
- *Data Type Conversion and Representation*
- *Other Methods*

---

### 3.1.1 Class Overview and Initialization

```
class bandhic.band_hic_matrix(contacts, diag_num=1, mask_row_col=None, mask=None, dtype=None,  
                             default_value=0, band_data_input=False)
```

Symmetric banded matrix stored in upper-triangular format. This storage format is motivated by high-resolution Hi-C data characteristics:

1. Symmetry of contact maps.
2. Interaction frequency concentrated near the diagonal; long-range contacts are sparse (mostly zero).
3. Contact frequency decays sharply with genomic distance.

By storing only the main and a fixed number of super-diagonals as columns of a band matrix (diagonal-major storage: diagonal  $k$  stored in column  $k$ ), we drastically reduce memory usage while enabling random access to Hi-C contacts. Additionally, `mask` and `mask_row_col` arrays track invalid or masked contacts to support downstream analysis.

Operations on this `band_hic_matrix` are as simple as on a `numpy.ndarray`; users can ignore these storage details.

This class stores only the main diagonal and up to  $(\text{diag\_num} - 1)$  super-diagonals, exploiting symmetry by mirroring values for lower-triangular access.

#### Parameters

- **contacts** (*Union[coo\_array, coo\_matrix, tuple, ndarray]*) –
- **diag\_num** (*int*) –
- **mask\_row\_col** (*Optional[ndarray]*) –
- **mask** (*Optional[Tuple[ndarray, ndarray]]*) –
- **dtype** (*Optional[type]*) –
- **default\_value** (*Union[int, float]*) –
- **band\_data\_input** (*bool*) –

#### shape

Shape of the original full Hi-C contact matrix (`bin_num, bin_num`), regardless of internal band storage format.

#### Type

tuple of int

#### dtype

Data type of the matrix elements, compatible with numpy dtypes.

#### Type

data-type

#### diag\_num

Number of diagonals stored.

#### Type

int

#### bin\_num

Number of bins (rows/columns) of the Hi-C matrix.

#### Type

int



---

**data**

Array of shape  $(bin\_num, diag\_num)$  storing banded Hi-C data.

**Type**

ndarray

**mask**

Mask for individual invalid entries. Stored as a boolean ndarray of shape  $(bin\_num, diag\_num)$  with the same shape as *data*.

**Type**

ndarray of bool or None

**mask\_row\_col**

Mask for entire rows and corresponding columns, indicating invalid bins. Stored as a boolean ndarray of shape  $(bin\_num,)$ . For computational convenience, row/column masks are also applied to the *mask* array to track masked entries.

**Type**

ndarray of bool or None

**default\_value**

Default value for out-of-band entries. Entries out of the banded region and not stored in the data array will be set to this value.

**Type**

scalar

## Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.shape
(4, 4)
```

```
__init__(contacts, diag_num=1, mask_row_col=None, mask=None, dtype=None, default_value=0,
         band_data_input=False)
```

Initialize a band\_hic\_matrix instance.

**Parameters**

- **contacts** (*{coo\_array, coo\_matrix, tuple, ndarray}*) – Input Hi-C data in COO format, tuple (data, (row, col)), full square array, or banded stored ndarray. For non-symmetric full arrays, only the upper-triangular part is used and the matrix is symmetrized. Full square arrays are not recommended for large matrices due to memory constraints.
- **diag\_num** (*int, optional*) – Number of diagonals to store. Must be  $\geq 1$  and  $\leq$  matrix dimension. Default is 1.
- **mask\_row\_col** (*ndarray of bool or indices, optional*) – Mask for invalid rows/columns. Can be specified as:
  - A boolean array of shape  $(bin\_num,)$  indicating which rows/columns to mask.
  - A list of indices to mask.Defaults to None (no masking).

- **mask** (*ndarray pair of (row\_indices, col\_indices), optional*) – Mask for invalid matrix entries. Can be specified as:
  - A tuple of two ndarray (row\_indices, col\_indices) listing positions to mask.
 Defaults to None (no masking).
- **dtype** (*data-type, optional*) – Desired numpy dtype; defaults to ‘contacts’ data dtype; compatible with numpy dtypes.
- **default\_value** (*scalar, optional*) – Default value for unstored out-of-band entries. Default is 0.
- **band\_data\_input** (*bool, optional*) – If True, contacts is treated as precomputed band storage. Default is False.

#### Raises

**ValueError** – If contacts type is invalid, diag\_num out of range, or array shape invalid.

#### Return type

None

### Examples

Initialize from a SciPy COO matrix:

```
>>> import bandhic as bh
>>> import numpy as np
>>> from scipy.sparse import coo_matrix
>>> coo = coo_matrix(([1, 2, 3], ([0, 1, 2],[0, 1, 2])), shape=(3,3))
>>> mat1 = bh.band_hic_matrix(coo, diag_num=2)
>>> mat1.data.shape
(3, 2)
```

Initialize from a tuple (data, (row, col)):

```
>>> mat2 = bh.band_hic_matrix(([4, 5, 6], ([0, 1, 2],[2, 1, 0])), diag_num=1)
>>> mat2.data.shape
(3, 1)
```

Initialize from a full dense array, only upper-triangular part is stored, lower part is symmetrized:

```
>>> arr = np.arange(16).reshape(4,4)
>>> mat3 = bh.band_hic_matrix(arr, diag_num=3)
>>> mat3.data.shape
(4, 3)
```

Initialize with row/column mask, this masks entire rows and corresponding columns:

```
>>> mask = np.array([True, False, False, True])
>>> mat4 = bh.band_hic_matrix(arr, diag_num=2, mask_row_col=mask)
>>> mat4.mask_row_col
array([ True, False, False,  True])
```

*mask\_row\_col* is also supported as a list of indices:

```
>>> mat4 = bh.band_hic_matrix(arr, diag_num=2, mask_row_col=[0, 3])
>>> mat4.mask_row_col
array([ True, False, False,  True])
```

Initialize from precomputed banded storage:

```
>>> band = mat3.data.copy()
>>> mat5 = bh.band_hic_matrix(band, band_data_input=True)
>>> mat5.data.shape
(4, 3)
```

### 3.1.2 Masking Methods

These methods allow for the application and manipulation of masks within the matrix. Masks can be used to exclude or highlight specific parts of the matrix during computations.

|                                                             |                                                                        |
|-------------------------------------------------------------|------------------------------------------------------------------------|
| <code>band_hic_matrix.init_mask()</code>                    | Initialize mask for invalid entries based on matrix shape.             |
| <code>band_hic_matrix.add_mask(row_idx, col_idx)</code>     | Add mask entries for specified indices.                                |
| <code>band_hic_matrix.get_mask()</code>                     | Get current mask array.                                                |
| <code>band_hic_matrix.unmask([indices])</code>              | Remove mask entries for specified indices or clear all.                |
| <code>band_hic_matrix.drop_mask()</code>                    | Clear the current mask by entry-level, but retain the row/column mask. |
| <code>band_hic_matrix.add_mask_row_col(mask_row_col)</code> | Mask entire rows and corresponding columns.                            |
| <code>band_hic_matrix.get_mask_row_col()</code>             | Get current row/column mask.                                           |
| <code>band_hic_matrix.drop_mask_row_col()</code>            | Clear the current row/column mask.                                     |
| <code>band_hic_matrix.clear_mask()</code>                   | Clear all masks (both entry-level and row/column-level).               |
| <code>band_hic_matrix.count_masked()</code>                 | Count the number of masked entries in the banded matrix.               |
| <code>band_hic_matrix.count_unmasked()</code>               | Count the number of unmasked entries in the banded matrix.             |
| <code>band_hic_matrix.count_in_band_masked()</code>         | Count the number of masked entries in the in-band region.              |
| <code>band_hic_matrix.count_out_band_masked()</code>        | Count the number of masked entries in the out-of-band region.          |
| <code>band_hic_matrix.count_in_band()</code>                | Count the number of valid entries in the in-band region.               |
| <code>band_hic_matrix.count_out_band()</code>               | Count the number of valid entries in the out-of-band region.           |

#### 3.1.2.1 bandhic.band\_hic\_matrix.init\_mask

`band_hic_matrix.init_mask()`

Initialize mask for invalid entries based on matrix shape.

**Raises**

**ValueError** – If mask is already initialized.

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.init_mask()
```

### 3.1.2.2 bandhic.band\_hic\_matrix.add\_mask

`band_hic_matrix.add_mask(row_idx, col_idx)`

Add mask entries for specified indices.

#### Parameters

- **row\_idx** (*array-like of int*) – Row indices to mask.
- **col\_idx** (*array-like of int*) – Column indices to mask.

#### Raises

**ValueError** – If `row_idx` and `col_idx` have different shapes.

#### Return type

None

## Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.add_mask([0, 1], [1, 2])
```

#### Return type

None

#### Parameters

- **row\_idx** (*Union[Sequence[int], ndarray]*) –
- **col\_idx** (*Union[Sequence[int], ndarray]*) –

### 3.1.2.3 bandhic.band\_hic\_matrix.get\_mask

`band_hic_matrix.get_mask()`

Get current mask array.

#### Returns

Current mask array or None if no mask.

#### Return type

ndarray or None

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mask = mat.get_mask()
```

### 3.1.2.4 bandhic.band\_hic\_matrix.unmask

`band_hic_matrix.unmask(indices=None)`

Remove mask entries for specified indices or clear all.

**Parameters**

**indices** (*tuple of array-like or None*) – Tuple (row\_idx, col\_idx) to unmask, or None to clear all.

**Raises**

**Warning** – If no mask exists when trying to remove.

**Return type**

None

## Notes

If *indices* is None, this will clear all masks (both entry-level and row/column), equivalent to *clear\_mask()*.

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.unmask(( [0], [0] ))
>>> mat.unmask()
```

**Return type**

None

**Parameters**

**indices** (*Optional[Tuple[ndarray, ndarray]]*) –

### 3.1.2.5 bandhic.band\_hic\_matrix.drop\_mask

`band_hic_matrix.drop_mask()`

Clear the current mask by entry-level, but retain the row/column mask.

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=(np.array([[0, 1], [1, 0],
↪ 2]])))
>>> mat.drop_mask()
```

## Notes

This method is useful when you want to keep the row/column mask but clear the entry-level mask. It will not affect the row/column mask, allowing you to maintain the masking of entire rows and columns.

### Return type

None

### 3.1.2.6 bandhic.band\_hic\_matrix.add\_mask\_row\_col

`band_hic_matrix.add_mask_row_col(mask_row_col)`

Mask entire rows and corresponding columns.

### Parameters

**mask\_row\_col** (*array-like of int or bool*) – If boolean array of shape (bin\_num,), True entries indicate rows/columns to mask. If integer array or sequence, treated as indices of rows/columns to mask.

### Raises

**ValueError** – If integer indices are out of bounds.

### Return type

None

## Examples

```
>>> import bandhic as bh
>>> mask = np.array([True, False, False])
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.add_mask_row_col(mask)
```

### Return type

None

### Parameters

**mask\_row\_col** (*Union[Sequence[int], Sequence[bool], ndarray, int]*) –

---

### 3.1.2.7 bandhic.band\_hic\_matrix.get\_mask\_row\_col

`band_hic_matrix.get_mask_row_col()`

Get current row/column mask.

**Returns**

Current row/column mask or None if no mask.

**Return type**

ndarray or None

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mask_row_col = mat.get_mask_row_col()
```

### 3.1.2.8 bandhic.band\_hic\_matrix.drop\_mask\_row\_col

`band_hic_matrix.drop_mask_row_col()`

Clear the current row/column mask.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.drop_mask_row_col()
```

### 3.1.2.9 bandhic.band\_hic\_matrix.clear\_mask

`band_hic_matrix.clear_mask()`

Clear all masks (both entry-level and row/column-level).

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.clear_mask()
```

**Return type**

None

---

### 3.1.2.10 bandhic.band\_hic\_matrix.count\_masked

`band_hic_matrix.count_masked()`

Count the number of masked entries in the banded matrix.

This counts the number of entries in the upper triangular part of the matrix, excluding the diagonal and valid entries.

**Returns**

Number of valid entries in the banded matrix.

**Return type**

int

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_masked()
0
```

### 3.1.2.11 bandhic.band\_hic\_matrix.count\_unmasked

`band_hic_matrix.count_unmasked()`

Count the number of unmasked entries in the banded matrix.

**Returns**

Number of unmasked entries in the banded matrix.

**Return type**

int

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_unmasked()
16
```

### 3.1.2.12 bandhic.band\_hic\_matrix.count\_in\_band\_masked

`band_hic_matrix.count_in_band_masked()`

Count the number of masked entries in the in-band region.

**Returns**

Number of masked entries in the in-band region.

**Return type**

int



---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_in_band_masked()
0
```

### 3.1.2.13 bandhic.band\_hic\_matrix.count\_out\_band\_masked

`band_hic_matrix.count_out_band_masked()`

Count the number of masked entries in the out-of-band region.

**Returns**

Number of masked entries in the out-of-band region.

**Return type**

int

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_out_band_masked()
0
```

### 3.1.2.14 bandhic.band\_hic\_matrix.count\_in\_band

`band_hic_matrix.count_in_band()`

Count the number of valid entries in the in-band region.

**Returns**

Number of valid entries in the in-band region.

**Return type**

int

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_in_band()
10
```

---

### 3.1.2.15 bandhic.band\_hic\_matrix.count\_out\_band

`band_hic_matrix.count_out_band()`

Count the number of valid entries in the out-of-band region.

**Returns**

Number of valid entries in the out-of-band region.

**Return type**

int

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_out_band()
6
```

## 3.1.3 Data Indexing and Modification

The following methods provide functionality to access, modify, and index the matrix data. These are essential for manipulating individual elements or subsets of the matrix.

|                                                                |                                                                        |
|----------------------------------------------------------------|------------------------------------------------------------------------|
| <code>band_hic_matrix.__getitem__(index)</code>                | Retrieve matrix entries or submatrix using NumPy-like indexing.        |
| <code>band_hic_matrix.__setitem__(index, values)</code>        | Assign values to matrix entries using NumPy-like indexing.             |
| <code>band_hic_matrix.get_values(row_idx, col_idx)</code>      | Retrieve values considering mask.                                      |
| <code>band_hic_matrix.set_values(row_idx, col_idx, ...)</code> | Set values at specified row and column indices.                        |
| <code>band_hic_matrix.diag(k)</code>                           | Retrieve the k-th diagonal from the matrix.                            |
| <code>band_hic_matrix.set_diag(k, values)</code>               | Set values in the k-th diagonal of the matrix.                         |
| <code>band_hic_matrix.extract_row(idx[, ...])</code>           | Extract stored, unmasked band values for a given row or column.        |
| <code>band_hic_matrix.__iter__()</code>                        | Iterate over diagonals of the matrix.                                  |
| <code>band_hic_matrix.iterwindows(width[, step])</code>        | Iterate over the diagonals of the matrix with a specified window size. |
| <code>band_hic_matrix.iterrows()</code>                        | Iterate over the rows of the band_hic_matrix object.                   |
| <code>band_hic_matrix.itercols()</code>                        | Iterate over the columns of the band_hic_matrix object.                |
| <code>band_hic_matrix.filled([fill_value, copy])</code>        | Fill masked entries in data with default value.                        |
| <code>band_hic_matrix.clip(min_val, max_val)</code>            | Clip data values to given range.                                       |

---

### 3.1.3.1 `bandhic.band_hic_matrix.__getitem__`

`band_hic_matrix.__getitem__(index)`

Retrieve matrix entries or submatrix using NumPy-like indexing.

Supports:

- Integer indexing: `mat[i, j]` returns a single value.
- Slice indexing: `mat[i:j, i:j]` returns a `band_hic_matrix` for square slices.
- Single-axis slice: `mat[i:j]` returns a `band_hic_matrix` same as `mat[i:j, i:j]`.
- Fancy (array) indexing: `mat[[i1, i2], [j1, j2]]` returns an ndarray or MaskedArray according to *mask*.
- Mixed indexing: combinations of integer, slice, and array-like indices.
- Boolean indexing: ‘`band_hic_matrix`’ object with dtype *bool* can be used to index entries.

When both row and column indices specify the same slice (or a single slice is provided), a new `band_hic_matrix` representing that square submatrix is returned. New submatrix is the view of the original matrix, sharing the same data and mask. If the mask or data is altered in the submatrix, the original matrix will reflect those changes as well.

- If a single integer index is provided for both row and column, a scalar value is returned.
- If a mask is set, masked entries will return as *numpy.ma.masked* for scalars, or as a *numpy.ma.MaskedArray* for arrays.
- If a mask is not set, the scalar value is returned directly, or a *numpy.ndarray* for arrays.
- If a square slice is provided, a new `band_hic_matrix` is returned with the same diagonal number and shape as the original matrix.
- If a single slice is provided, it returns a `band_hic_matrix` with the same diagonal number and shape as the original matrix.
- If fancy indexing is used, it returns a *numpy.ndarray* or *numpy.ma.MaskedArray* depending on whether the mask is set.
- In all other cases, a *numpy.ndarray* (if no mask) or *numpy.ma.MaskedArray* (if mask present) is returned.

#### Parameters

**index** (*int*, *slice*, *band\_hic\_matrix*, *array-like of int*, or *tuple of these*)

– index expression for rows and columns. May be:

- A pair (*row\_idx*, *col\_idx*) of ints, slices, or array-like for mixed indexing.
- A single slice selecting a square region.
- A *band\_hic\_matrix* object with dtype of *bool* for boolean indexing.

#### Returns

- scalar : when both row and column are integer indices.
- *numpy.ndarray* : for fancy or mixed indexing without mask.
- *numpy.ma.MaskedArray* : for fancy or mixed indexing when mask is set.
- `band_hic_matrix` : for square slice results.

#### Return type

scalar or ndarray or MaskedArray or *band\_hic\_matrix*

---

## Raises

**ValueError** – If a slice step is not 1, or if indices are out of bounds.

## Examples

```
>>> import numpy as np
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(16).reshape(4,4), diag_num=2)
```

# Single-element access (scalar)

```
>>> mat[1, 2]
np.int64(6)
```

# Masked element returns masked

```
>>> mat2 = bh.band_hic_matrix(np.eye(4), dtype=int, diag_num=2, mask=([0],[1]))
>>> mat2[0, 1]
masked
```

# Square submatrix via two-slice indexing returns band\_hic\_matrix

```
>>> sub = mat[1:3, 1:3]
>>> isinstance(sub, bh.band_hic_matrix)
True
```

# Single-axis slice returns band\_hic\_matrix for square region

```
>>> sub2 = mat[0:2] # equivalent to mat[0:2, 0:2]
>>> isinstance(sub2, bh.band_hic_matrix)
True
```

# Fancy indexing returns ndarray or MaskedArray

```
>>> arr = mat[[0,2,3], [1,2,0]]
>>> isinstance(arr, np.ndarray)
True
```

```
>>> mat.add_mask([0,1],[1,2]) # Add mask to some entries
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True
```

# Boolean indexing with band\_hic\_matrix

```
>>> mat3 = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=([0,1],[1,2]))
>>> bool_mask = mat3 > 0 # Create a boolean mask
>>> result = mat3[bool_mask] # Use boolean mask for indexing
>>> isinstance(result, np.ma.MaskedArray)
True
```

```
>>> result
masked_array(data=[1.0, 1.0, 1.0, 1.0],
```

(continues on next page)

```
mask=[False, False, False, False],
fill_value=0.0)
```

### 3.1.3.2 bandhic.band\_hic\_matrix.\_\_setitem\_\_

`band_hic_matrix.__setitem__(index, values)`

Assign values to matrix entries using NumPy-like indexing.

#### Parameters

- **index** (*int, tuple of (row\_idx, col\_idx), slice, or band\_hic\_matrix*) – Index expression for rows and columns. May be:
- **column.** (*- A single integer for both row and*) –
- **int** (*- A tuple of row and column indices (can be)*) –
- **slice** –
- **array-like).** (*or*) –
- **region.** (*- A single slice selecting a square*) –
- **indexing.** (*- A band\_hic\_matrix object with dtype of bool for boolean*) –
- **values** (*scalar or array-like*) – Values to assign. Can be a single scalar or an array-like object.

#### Raises

- **ValueError** – If index is a slice with step not equal to 1, or if indices exceed matrix dimensions.
- **TypeError** – If *values* is not a scalar or array-like object.
- **Supports:** –
  - - **Integer indexing** – `mat[i, j] = value` assigns to a single element.:
  - - **Slice indexing** – `mat[i:j, i:j] = array or scalar` assigns to a square submatrix.:
  - - **Single-axis slice** – `mat[i:j] = ...` is equivalent to `mat[i:j, i:j].:`
  - - **Fancy (array) indexing** – `mat[[i1, i2], [j1, j2]] = array or scalar` for scattered assignments.:
  - - **Mixed indexing** – combinations of integer, slice, and array-like indices.:
  - - **Boolean indexing** – boolean mask (another `band_hic_matrix` with `dtype=bool`) selects entries to set.:

#### Return type

None

---

## Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.zeros((4,4)), diag_num=2, dtype=int)
```

# Single element assignment

```
>>> mat[1, 2] = 5
>>> mat[1, 2]
5
```

# Slice assignment to square submatrix

```
>>> mat[0:2, 0:2] = [[1, 2], [2, 4]]
>>> mat[0:2, 0:2].todense()
array([[1, 2],
       [2, 4]])
```

# Single-axis slice assignment (equivalent square slice)

```
>>> mat[2:4] = 0
>>> mat[2:4].todense()
array([[0, 0],
       [0, 0]])
```

# Fancy indexing for scattered assignments

```
>>> mat[[0, 3], [1, 2]] = [7, 8]
>>> mat[0, 1], mat[3, 2]
(7, 8)
```

# Boolean mask assignment

```
>>> mat2 = bh.band_hic_matrix(np.eye(4), diag_num=2, dtype=int)
>>> bool_mask = mat2 > 0
>>> mat2[bool_mask] = 9
>>> mat2.todense()
array([[9, 0, 0, 0],
       [0, 9, 0, 0],
       [0, 0, 9, 0],
       [0, 0, 0, 9]])
```

## Notes

- Assigning to masked entries updates underlying data but does not automatically unmask.
- For multidimensional assignments, scalar values broadcast to all selected positions, while array values must match the number of targeted elements.
- If a boolean mask is used, it must be a *band\_hic\_matrix* with dtype *bool* and the same shape as the original matrix.
- If a single slice is provided, it behaves like `mat[i:j, i:j]` for square submatrices.

---

**Return type**

None

**Parameters**

- **index** (*Union[Tuple[Union[int, slice, ndarray, list], Union[int, slice, ndarray, list]], slice, band\_hic\_matrix*) –
- **values** (*Union[Number, ndarray, list]*) –

### 3.1.3.3 bandhic.band\_hic\_matrix.get\_values

`band_hic_matrix.get_values(row_idx, col_idx)`

Retrieve values considering mask.

**Parameters**

- **row\_idx** (*array-like of int*) – Row indices.
- **col\_idx** (*array-like of int*) – Column indices.

**Returns**

Retrieved values; masked entries yield masked results.

**Return type**

ndarray or MaskedArray

**Raises**

**ValueError** – If indices exceed matrix dimensions.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.get_values([0,1],[1,2])
array([1., 1.]
```

### 3.1.3.4 bandhic.band\_hic\_matrix.set\_values

`band_hic_matrix.set_values(row_idx, col_idx, values)`

Set values at specified row and column indices.

**Parameters**

- **row\_idx** (*array-like of int*) – Row indices where values will be set.
- **col\_idx** (*array-like of int*) – Column indices where values will be set.
- **values** (*scalar or array-like*) – Values to assign.

**Return type**

None

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((3,3), diag_num=2, dtype=int)
>>> mat.set_values([0,1], [1,2], [4,5])
>>> mat[0,1]
4
```

## Notes

Writing to masked positions will update the underlying data but will not clear the mask.

### Return type

None

### Parameters

- **row\_idx** (*ndarray*) –
- **col\_idx** (*ndarray*) –
- **values** (*Union[Number, ndarray]*) –

### 3.1.3.5 bandhic.band\_hic\_matrix.diag

`band_hic_matrix.diag(k)`

Retrieve the *k*-th diagonal from the matrix.

### Parameters

**k** (*int*) – The diagonal index to retrieve.

### Returns

The *k*-th diagonal values; masked if mask is set.

### Return type

*ndarray* or *MaskedArray*

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.diag(1)
array([1., 1.])
```



---

### 3.1.3.6 bandhic.band\_hic\_matrix.set\_diag

`band_hic_matrix.set_diag(k, values)`

Set values in the k-th diagonal of the matrix.

**Parameters**

- **k** (*int*) – The diagonal index to set.
- **values** (*array-like*) – The values to set in the diagonal.

**Raises**

**ValueError** – If k is out of range or values length mismatch.

**Return type**

None

### Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((4,4), diag_num=3)
>>> mat.set_diag(1, [9,9,9])
>>> mat.diag(1)
array([9., 9., 9.])
```

**Return type**

None

**Parameters**

- **k** (*int*) –
- **values** (*Iterable[Any]*) –

### 3.1.3.7 bandhic.band\_hic\_matrix.extract\_row

`band_hic_matrix.extract_row(idx, extract_out_of_band=True)`

Extract stored, unmasked band values for a given row or column.

**Parameters**

- **idx** (*int*) – Row (or column, due to symmetry) index for which to extract band values.
- **extract\_out\_of\_band** (*bool, optional*) – If True, include out-of-band entries filled with *default\_value* and masked if appropriate. If False (default), return only the stored band values.

**Returns**

If *extract\_out\_of\_band=False*, a 1D array of length up to *diag\_num* containing band values. If *extract\_out\_of\_band=True*, a 1D array of length *bin\_num* with all row/column values.

**Return type**

ndarray or MaskedArray

**Raises**

**ValueError** – If *idx* is outside the range [0, bin\_num-1].

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.extract_row(0, extract_out_of_band=False)
array([0, 1])
>>> mat.extract_row(0)
array([0, 1, 0])
```

### 3.1.3.8 bandhic.band\_hic\_matrix.\_\_iter\_\_

`band_hic_matrix.__iter__()`

Iterate over diagonals of the matrix.

#### Yields

*ndarray* – Values of each diagonal.

#### Return type

*Iterator[ndarray]*

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for band in mat:
...     print(band)
[1. 1. 1.]
[1. 1.]
```

#### Return type

*Iterator[ndarray]*

### 3.1.3.9 bandhic.band\_hic\_matrix.iterwindows

`band_hic_matrix.iterwindows(width, step=1)`

Iterate over the diagonals of the matrix with a specified window size.

#### Parameters

- **width** (*int*) – The size of the window to iterate over.
- **step** (*int, optional*) – Step size between windows. Default is 1.

#### Yields

*band\_hic\_matrix* – The values in the current window.

#### Return type

*Iterator[band\_hic\_matrix]*

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for win in mat.iterwindows(2):
...     print(win)
band_hic_matrix(shape=(2, 2), diag_num=2, dtype=<class 'numpy.float64'>)
band_hic_matrix(shape=(2, 2), diag_num=2, dtype=<class 'numpy.float64'>)
```

### Return type

Iterator[[band\\_hic\\_matrix](#)]

### Parameters

- **width** (*int*) –
- **step** (*int*) –

### 3.1.3.10 `bandhic.band_hic_matrix.iterrows`

`band_hic_matrix.iterrows()`

Iterate over the rows of the `band_hic_matrix` object.

### Yields

*ndarray* – The values in the current row.

### Return type

Iterator[*ndarray*]

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for row in mat.iterrows():
...     print(row)
[1. 1. 1.]
[1. 1. 1.]
[1. 1. 1.]
```

### Return type

Iterator[*ndarray*]

### 3.1.3.11 `bandhic.band_hic_matrix.itercols`

`band_hic_matrix.itercols()`

Iterate over the columns of the `band_hic_matrix` object.

### Yields

*ndarray* – The values in the current column.

### Return type

Iterator[*ndarray*]

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for col in mat.itercols():
...     print(col)
[1. 1. 1.]
[1. 1. 1.]
[1. 1. 1.]
```

### Return type

Iterator[ndarray]

### 3.1.3.12 bandhic.band\_hic\_matrix.filled

`band_hic_matrix.filled(fill_value=None, copy=True)`

Fill masked entries in data with default value.

#### Parameters

- **fill\_value** (*scalar*) – Value to assign to masked entries.
- **copy** (*bool*) –

#### Raises

**ValueError** – If no mask is initialized.

#### Return type

`band_hic_matrix`

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2, mask = ([0,1],[1,2]))
>>> mat.filled()
band_hic_matrix(shape=(4, 4), diag_num=2, dtype=float64)
```

### Return type

`band_hic_matrix`

#### Parameters

- **fill\_value** (*Optional[Union[int, float]]*) –
- **copy** (*bool*) –

---

### 3.1.3.13 bandhic.band\_hic\_matrix.clip

`band_hic_matrix.clip(min_val, max_val)`

Clip data values to given range.

#### Parameters

- **min\_val** (*scalar*) –
- **max\_val** (*scalar*) –

#### Return type

None

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat = mat.clip(0, 10)
```

#### Return type

None

#### Parameters

- **min\_val** (*Number*) –
- **max\_val** (*Number*) –

## 3.1.4 Data Reduction and Normalization

These methods enable various data reduction operations, such as summing, averaging, or aggregating matrix values along specific axes or dimensions.

---

|                                                       |                                                                                         |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>band_hic_matrix.min([axis])</code>              | Compute the minimum value in the matrix or along a given axis.                          |
| <code>band_hic_matrix.max([axis])</code>              | Compute the maximum value in the matrix or along a given axis.                          |
| <code>band_hic_matrix.sum([axis])</code>              | Compute the sum of the values in the matrix or along a given axis.                      |
| <code>band_hic_matrix.mean([axis])</code>             | Compute the mean value of the matrix or along a given axis.                             |
| <code>band_hic_matrix.std([axis])</code>              | Compute the standard deviation of the values in the matrix or along a given axis.       |
| <code>band_hic_matrix.var([axis])</code>              | Compute the variance of the values in the matrix or along a given axis.                 |
| <code>band_hic_matrix.prod([axis])</code>             | Compute the product of the values in the matrix or along a given axis.                  |
| <code>band_hic_matrix.ptp([axis])</code>              | Compute the peak-to-peak (maximum - minimum) value of the matrix or along a given axis. |
| <code>band_hic_matrix.all([axis, banded_only])</code> | Test whether all (or any) array elements along a given axis evaluate to True.           |
| <code>band_hic_matrix.any([axis, banded_only])</code> | Test whether all (or any) array elements along a given axis evaluate to True.           |
| <code>band_hic_matrix.normalize([inplace])</code>     | Normalize each diagonal of the matrix to have zero mean and unit variance.              |

---

### 3.1.4.1 bandhic.band\_hic\_matrix.min

`band_hic_matrix.min(axis=None)`

Compute the minimum value in the matrix or along a given axis.

#### Parameters

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the minimum:

- None: compute over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

#### Returns

Minimum value(s) along the specified axis.

#### Return type

scalar or ndarray

#### Raises

**ValueError** – If axis is not one of the supported values.

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.min() # global minimum
0
>>> mat.min(axis='row')
array([0, 1, 0])
```

### 3.1.4.2 bandhic.band\_hic\_matrix.max

`band_hic_matrix.max(axis=None)`

Compute the maximum value in the matrix or along a given axis.

#### Parameters

**axis** (*None*, *int*, or *{'row','col','diag'}*, *optional*) – Axis along which to compute the maximum:

- *None*: compute over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is *None*.

#### Returns

Maximum value(s) along the specified axis.

#### Return type

scalar or ndarray

#### Raises

**ValueError** – If axis is not one of the supported values.

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.max() # global maximum
8
>>> mat.max(axis='row')
array([1, 5, 8])
```

---

### 3.1.4.3 bandhic.band\_hic\_matrix.sum

`band_hic_matrix.sum(axis=None)`

Compute the sum of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the sum:

- *None*: sum over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is *None*.

**Returns**

Sum(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.sum() # sum of all elements
24
>>> mat.sum(axis='row')
array([ 1, 10, 13])
```

### 3.1.4.4 bandhic.band\_hic\_matrix.mean

`band_hic_matrix.mean(axis=None)`

Compute the mean value of the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the mean:

- *None*: mean over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is *None*.

**Returns**

Mean value(s) along the specified axis.



---

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.mean() # mean of all elements
2.6666666666666665
>>> mat.mean(axis='diag')
array([4., 3.] )
```

### 3.1.4.5 bandhic.band\_hic\_matrix.std

`band_hic_matrix.std(axis=None)`

Compute the standard deviation of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row', 'col', 'diag'}, optional*) – Axis along which to compute the standard deviation:

- None: std over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Standard deviation(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.std() # std of all elements
2.748737083745107
>>> mat.std(axis='diag')
array([3.26598632, 2.        ] )
```

---

### 3.1.4.6 bandhic.band\_hic\_matrix.var

`band_hic_matrix.var(axis=None)`

Compute the variance of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the variance:

- None: variance over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Variance(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.var() # variance of all elements
7.555555555555555
>>> mat.var(axis='row')
array([ 0.22222222,  2.88888889, 10.88888889])
```

### 3.1.4.7 bandhic.band\_hic\_matrix.prod

`band_hic_matrix.prod(axis=None)`

Compute the product of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the product:

- None: product over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Product(s) along the specified axis.

---

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(1, 10).reshape(3,3), diag_num=2)
>>> mat.prod() # product of all elements
0
>>> mat.prod(axis='row')
array([ 0, 60, 0])
```

**3.1.4.8 bandhic.band\_hic\_matrix.ptp**

`band_hic_matrix.ptp(axis=None)`

Compute the peak-to-peak (maximum - minimum) value of the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the peak-to-peak value:

- None: ptp over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Peak-to-peak value(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.ptp() # ptp of all elements
8
>>> mat.ptp(axis='row')
array([1, 4, 8])
```

---

### 3.1.4.9 bandhic.band\_hic\_matrix.all

`band_hic_matrix.all(axis=None, banded_only=False)`

Test whether all (or any) array elements along a given axis evaluate to True.

#### Parameters

- **axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to test.
  - None: test all stored values (and default for missing).
  - 0 or 'row': per-row reduction.
  - 1 or 'col': per-column reduction.
  - 'diag': per-diagonal reduction.
- **banded\_only** (*bool, optional*) – If True, only consider stored band elements; ignore out-of-band values. Default is False.

#### Returns

Boolean result(s) of the test.

#### Return type

bool or ndarray

#### Raises

**ValueError** – If axis is not supported.

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.all()
False
>>> mat.any(axis='diag', banded_only=True)
array([ True, False])
```

### 3.1.4.10 bandhic.band\_hic\_matrix.any

`band_hic_matrix.any(axis=None, banded_only=False)`

Test whether all (or any) array elements along a given axis evaluate to True.

#### Parameters

- **axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to test:
  - None: test all stored values (and default for missing).
  - 0 or 'row': per-row reduction.
  - 1 or 'col': per-column reduction.
  - 'diag': per-diagonal reduction.
- **banded\_only** (*bool, optional*) – If True, only consider stored band elements; ignore out-of-band values. Default is False.

#### Returns

Boolean result(s) of the test.

---

**Return type**

bool or ndarray

**Raises**

**ValueError** – If axis is not supported.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.all()
False
>>> mat.any(axis='diag', banded_only=True)
array([ True, False])
```

### 3.1.4.11 bandhic.band\_hic\_matrix.normalize

`band_hic_matrix.normalize(inplace=False)`

Normalize each diagonal of the matrix to have zero mean and unit variance. This modifies the matrix in place.  
:raises UserWarning: If any diagonal has zero standard deviation, it will be set to zero.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat = mat.normalize()
```

**Return type**

None

**Parameters**

**inplace** (*bool*) –

## 3.1.5 Vectorized Computation Methods

These methods allow for the application of universal functions (ufuncs) to the matrix, enabling element-wise operations similar to those in NumPy.

|                                                                |                                                           |
|----------------------------------------------------------------|-----------------------------------------------------------|
| <code>band_hic_matrix.absolute(self, *args, **kwargs)</code>   | Perform element-wise 'absolute' operation.                |
| <code>band_hic_matrix.add(self, other, *args, **kwargs)</code> | Perform element-wise 'add' operation with two inputs.     |
| <code>band_hic_matrix.arccos(self, *args, **kwargs)</code>     | Perform element-wise 'arccos' operation.                  |
| <code>band_hic_matrix.arccosh(self, *args, **kwargs)</code>    | Perform element-wise 'arccosh' operation.                 |
| <code>band_hic_matrix.arcsin(self, *args, **kwargs)</code>     | Perform element-wise 'arcsin' operation.                  |
| <code>band_hic_matrix.arcsinh(self, *args, **kwargs)</code>    | Perform element-wise 'arcsinh' operation.                 |
| <code>band_hic_matrix.arctan(self, *args, **kwargs)</code>     | Perform element-wise 'arctan' operation.                  |
| <code>band_hic_matrix.arctan2(self, other, *args, ...)</code>  | Perform element-wise 'arctan2' operation with two inputs. |
| <code>band_hic_matrix.arctanh(self, *args, **kwargs)</code>    | Perform element-wise 'arctanh' operation.                 |

continues on next page

Table 3.1 – continued from previous page

|                                                                |                                                                 |
|----------------------------------------------------------------|-----------------------------------------------------------------|
| <code>band_hic_matrix.bitwise_and(self, other, ...)</code>     | Perform element-wise 'bitwise_and' operation with two inputs.   |
| <code>band_hic_matrix.bitwise_or(self, other, ...)</code>      | Perform element-wise 'bitwise_or' operation with two inputs.    |
| <code>band_hic_matrix.bitwise_xor(self, other, ...)</code>     | Perform element-wise 'bitwise_xor' operation with two inputs.   |
| <code>band_hic_matrix.cbrt(self, *args, **kwargs)</code>       | Perform element-wise 'cbrt' operation.                          |
| <code>band_hic_matrix.conj(self, *args, **kwargs)</code>       | Perform element-wise 'conj' operation.                          |
| <code>band_hic_matrix.conjugate(self, *args, **kwargs)</code>  | Perform element-wise 'conjugate' operation.                     |
| <code>band_hic_matrix.cos(self, *args, **kwargs)</code>        | Perform element-wise 'cos' operation.                           |
| <code>band_hic_matrix.cosh(self, *args, **kwargs)</code>       | Perform element-wise 'cosh' operation.                          |
| <code>band_hic_matrix.deg2rad(self, *args, **kwargs)</code>    | Perform element-wise 'deg2rad' operation.                       |
| <code>band_hic_matrix.degrees(self, *args, **kwargs)</code>    | Perform element-wise 'degrees' operation.                       |
| <code>band_hic_matrix.divide(self, other, *args, ...)</code>   | Perform element-wise 'divide' operation with two inputs.        |
| <code>band_hic_matrix.divmod(self, other, *args, ...)</code>   | Perform element-wise 'divmod' operation with two inputs.        |
| <code>band_hic_matrix.equal(self, other, *args, ...)</code>    | Perform element-wise 'equal' operation with two inputs.         |
| <code>band_hic_matrix.exp(self, *args, **kwargs)</code>        | Perform element-wise 'exp' operation.                           |
| <code>band_hic_matrix.exp2(self, *args, **kwargs)</code>       | Perform element-wise 'exp2' operation.                          |
| <code>band_hic_matrix.expm1(self, *args, **kwargs)</code>      | Perform element-wise 'expm1' operation.                         |
| <code>band_hic_matrix.fabs(self, *args, **kwargs)</code>       | Perform element-wise 'fabs' operation.                          |
| <code>band_hic_matrix.float_power(self, other, ...)</code>     | Perform element-wise 'float_power' operation with two inputs.   |
| <code>band_hic_matrix.floor_divide(self, other, ...)</code>    | Perform element-wise 'floor_divide' operation with two inputs.  |
| <code>band_hic_matrix.fmod(self, other, *args, ...)</code>     | Perform element-wise 'fmod' operation with two inputs.          |
| <code>band_hic_matrix.gcd(self, other, *args, **kwargs)</code> | Perform element-wise 'gcd' operation with two inputs.           |
| <code>band_hic_matrix.greater(self, other, *args, ...)</code>  | Perform element-wise 'greater' operation with two inputs.       |
| <code>band_hic_matrix.greater_equal(self, other, ...)</code>   | Perform element-wise 'greater_equal' operation with two inputs. |
| <code>band_hic_matrix.heaviside(self, other, ...)</code>       | Perform element-wise 'heaviside' operation with two inputs.     |
| <code>band_hic_matrix.hypot(self, other, *args, ...)</code>    | Perform element-wise 'hypot' operation with two inputs.         |
| <code>band_hic_matrix.invert(self, *args, **kwargs)</code>     | Perform element-wise 'invert' operation.                        |
| <code>band_hic_matrix.lcm(self, other, *args, **kwargs)</code> | Perform element-wise 'lcm' operation with two inputs.           |
| <code>band_hic_matrix.left_shift(self, other, ...)</code>      | Perform element-wise 'left_shift' operation with two inputs.    |
| <code>band_hic_matrix.less(self, other, *args, ...)</code>     | Perform element-wise 'less' operation with two inputs.          |
| <code>band_hic_matrix.less_equal(self, other, ...)</code>      | Perform element-wise 'less_equal' operation with two inputs.    |
| <code>band_hic_matrix.log(self, *args, **kwargs)</code>        | Perform element-wise 'log' operation.                           |
| <code>band_hic_matrix.log1p(self, *args, **kwargs)</code>      | Perform element-wise 'log1p' operation.                         |
| <code>band_hic_matrix.log2(self, *args, **kwargs)</code>       | Perform element-wise 'log2' operation.                          |
| <code>band_hic_matrix.log10(self, *args, **kwargs)</code>      | Perform element-wise 'log10' operation.                         |
| <code>band_hic_matrix.logaddexp(self, other, ...)</code>       | Perform element-wise 'logaddexp' operation with two inputs.     |
| <code>band_hic_matrix.logaddexp2(self, other, ...)</code>      | Perform element-wise 'logaddexp2' operation with two inputs.    |
| <code>band_hic_matrix.logical_and(self, other, ...)</code>     | Perform element-wise 'logical_and' operation with two inputs.   |

continues on next page

Table 3.1 – continued from previous page

|                                                                |                                                               |
|----------------------------------------------------------------|---------------------------------------------------------------|
| <code>band_hic_matrix.logical_or(self, other, ...)</code>      | Perform element-wise 'logical_or' operation with two inputs.  |
| <code>band_hic_matrix.logical_xor(self, other, ...)</code>     | Perform element-wise 'logical_xor' operation with two inputs. |
| <code>band_hic_matrix.maximum(self, other, *args, ...)</code>  | Perform element-wise 'maximum' operation with two inputs.     |
| <code>band_hic_matrix.minimum(self, other, *args, ...)</code>  | Perform element-wise 'minimum' operation with two inputs.     |
| <code>band_hic_matrix.mod(self, other, *args, **kwargs)</code> | Perform element-wise 'mod' operation with two inputs.         |
| <code>band_hic_matrix.multiply(self, other, *args, ...)</code> | Perform element-wise 'multiply' operation with two inputs.    |
| <code>band_hic_matrix.negative(self, *args, **kwargs)</code>   | Perform element-wise 'negative' operation.                    |
| <code>band_hic_matrix.not_equal(self, other, ...)</code>       | Perform element-wise 'not_equal' operation with two inputs.   |
| <code>band_hic_matrix.positive(self, *args, **kwargs)</code>   | Perform element-wise 'positive' operation.                    |
| <code>band_hic_matrix.power(self, other, *args, ...)</code>    | Perform element-wise 'power' operation with two inputs.       |
| <code>band_hic_matrix.rad2deg(self, *args, **kwargs)</code>    | Perform element-wise 'rad2deg' operation.                     |
| <code>band_hic_matrix.radians(self, *args, **kwargs)</code>    | Perform element-wise 'radians' operation.                     |
| <code>band_hic_matrix.reciprocal(self, *args, **kwargs)</code> | Perform element-wise 'reciprocal' operation.                  |
| <code>band_hic_matrix.remainder(self, other, ...)</code>       | Perform element-wise 'remainder' operation with two inputs.   |
| <code>band_hic_matrix.right_shift(self, other, ...)</code>     | Perform element-wise 'right_shift' operation with two inputs. |
| <code>band_hic_matrix rint(self, *args, **kwargs)</code>       | Perform element-wise 'rint' operation.                        |
| <code>band_hic_matrix.sign(self, *args, **kwargs)</code>       | Perform element-wise 'sign' operation.                        |
| <code>band_hic_matrix.sin(self, *args, **kwargs)</code>        | Perform element-wise 'sin' operation.                         |
| <code>band_hic_matrix.sinh(self, *args, **kwargs)</code>       | Perform element-wise 'sinh' operation.                        |
| <code>band_hic_matrix.sqrt(self, *args, **kwargs)</code>       | Perform element-wise 'sqrt' operation.                        |
| <code>band_hic_matrix.square(self, *args, **kwargs)</code>     | Perform element-wise 'square' operation.                      |
| <code>band_hic_matrix.subtract(self, other, *args, ...)</code> | Perform element-wise 'subtract' operation with two inputs.    |
| <code>band_hic_matrix.tan(self, *args, **kwargs)</code>        | Perform element-wise 'tan' operation.                         |
| <code>band_hic_matrix.tanh(self, *args, **kwargs)</code>       | Perform element-wise 'tanh' operation.                        |
| <code>band_hic_matrix.true_divide(self, other, ...)</code>     | Perform element-wise 'true_divide' operation with two inputs. |

### 3.1.5.1 bandhic.band\_hic\_matrix.absolute

`band_hic_matrix.absolute(self, *args, **kwargs)`

Perform element-wise 'absolute' operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.absolute`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.absolute`.

#### Returns

Result of element-wise 'absolute' operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.absolute`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.absolute()
```

### 3.1.5.2 bandhic.band\_hic\_matrix.add

`band_hic_matrix.add(self, other, *args, **kwargs)`

Perform element-wise ‘add’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.add`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.add`.

**Returns**

Result of element-wise ‘add’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.add`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.add(other)
```

### 3.1.5.3 bandhic.band\_hic\_matrix.arccos

`band_hic_matrix.arccos(self, *args, **kwargs)`

Perform element-wise ‘arccos’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arccos`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arccos`.



---

**Returns**

Result of element-wise ‘arccos’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arccos`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arccos()
```

### 3.1.5.4 bandhic.band\_hic\_matrix.arccosh

`band_hic_matrix.arccosh(self, *args, **kwargs)`

Perform element-wise ‘arccosh’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arccosh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arccosh`.

**Returns**

Result of element-wise ‘arccosh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arccosh`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arccosh()
```

### 3.1.5.5 bandhic.band\_hic\_matrix.arcsin

`band_hic_matrix.arcsin(self, *args, **kwargs)`

Perform element-wise ‘arcsin’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arcsin`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arcsin`.

---

**Returns**

Result of element-wise ‘arcsin’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arcsin`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arcsin()
```

### 3.1.5.6 bandhic.band\_hic\_matrix.arcsinh

`band_hic_matrix.arcsinh(self, *args, **kwargs)`

Perform element-wise ‘arcsinh’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arcsinh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arcsinh`.

**Returns**

Result of element-wise ‘arcsinh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arcsinh`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arcsinh()
```

### 3.1.5.7 bandhic.band\_hic\_matrix.arctan

`band_hic_matrix.arctan(self, *args, **kwargs)`

Perform element-wise ‘arctan’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arctan`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arctan`.

---

**Returns**

Result of element-wise ‘arctan’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arctan`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arctan()
```

### 3.1.5.8 bandhic.band\_hic\_matrix.arctan2

`band_hic_matrix.arctan2(self, other, *args, **kwargs)`

Perform element-wise ‘arctan2’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arctan2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arctan2`.

**Returns**

Result of element-wise ‘arctan2’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arctan2`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.arctan2(other)
```

---

### 3.1.5.9 bandhic.band\_hic\_matrix.arctanh

`band_hic_matrix.arctanh(self, *args, **kwargs)`

Perform element-wise ‘arctanh’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.arctanh`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.arctanh`.

#### Returns

Result of element-wise ‘arctanh’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.arctanh`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arctanh()
```

### 3.1.5.10 bandhic.band\_hic\_matrix.bitwise\_and

`band_hic_matrix.bitwise_and(self, other, *args, **kwargs)`

Perform element-wise ‘bitwise\_and’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.bitwise_and`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.bitwise_and`.

#### Returns

Result of element-wise ‘bitwise\_and’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.bitwise_and`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_and(other)
```

### 3.1.5.11 bandhic.band\_hic\_matrix.bitwise\_or

`band_hic_matrix.bitwise_or(self, other, *args, **kwargs)`

Perform element-wise ‘bitwise\_or’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.bitwise_or`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.bitwise_or`.

#### Returns

Result of element-wise ‘bitwise\_or’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.bitwise_or`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_or(other)
```

### 3.1.5.12 bandhic.band\_hic\_matrix.bitwise\_xor

`band_hic_matrix.bitwise_xor(self, other, *args, **kwargs)`

Perform element-wise ‘bitwise\_xor’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.bitwise_xor`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.bitwise_xor`.

#### Returns

Result of element-wise ‘bitwise\_xor’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.bitwise_xor`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_xor(other)
```

**3.1.5.13 bandhic.band\_hic\_matrix.cbrt**

`band_hic_matrix.cbrt(self, *args, **kwargs)`

Perform element-wise ‘cbrt’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cbrt`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cbrt`.

**Returns**

Result of element-wise ‘cbrt’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.cbrt`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cbrt()
```

**3.1.5.14 bandhic.band\_hic\_matrix.conj**

`band_hic_matrix.conj(self, *args, **kwargs)`

Perform element-wise ‘conj’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.conj`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.conj`.

---

**Returns**

Result of element-wise ‘conj’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.conj`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.conj()
```

### 3.1.5.15 bandhic.band\_hic\_matrix.conjugate

`band_hic_matrix.conjugate(self, *args, **kwargs)`

Perform element-wise ‘conjugate’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.conjugate`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.conjugate`.

**Returns**

Result of element-wise ‘conjugate’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.conjugate`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.conjugate()
```

### 3.1.5.16 bandhic.band\_hic\_matrix.cos

`band_hic_matrix.cos(self, *args, **kwargs)`

Perform element-wise ‘cos’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cos`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cos`.

---

**Returns**

Result of element-wise ‘cos’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

[numpy.cos](#)

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cos()
```

### 3.1.5.17 bandhic.band\_hic\_matrix.cosh

`band_hic_matrix.cosh(self, *args, **kwargs)`

Perform element-wise ‘cosh’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cosh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cosh`.

**Returns**

Result of element-wise ‘cosh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

[numpy.cosh](#)

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cosh()
```

### 3.1.5.18 bandhic.band\_hic\_matrix.deg2rad

`band_hic_matrix.deg2rad(self, *args, **kwargs)`

Perform element-wise ‘deg2rad’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.deg2rad`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.deg2rad`.



---

**Returns**

Result of element-wise ‘deg2rad’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.deg2rad`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.deg2rad()
```

### 3.1.5.19 bandhic.band\_hic\_matrix.degrees

`band_hic_matrix.degrees(self, *args, **kwargs)`

Perform element-wise ‘degrees’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.degrees`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.degrees`.

**Returns**

Result of element-wise ‘degrees’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.degrees`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.degrees()
```

### 3.1.5.20 bandhic.band\_hic\_matrix.divide

`band_hic_matrix.divide(self, other, *args, **kwargs)`

Perform element-wise ‘divide’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix or array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.divide`.

- 
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.divide`.

**Returns**

Result of element-wise ‘divide’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.divide`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.divide(other)
```

### 3.1.5.21 `bandhic.band_hic_matrix.divmod`

`band_hic_matrix.divmod(self, other, *args, **kwargs)`

Perform element-wise ‘divmod’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.divmod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.divmod`.

**Returns**

Result of element-wise ‘divmod’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.divmod`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.divmod(other)
```

---

### 3.1.5.22 `bandhic.band_hic_matrix.equal`

`band_hic_matrix.equal(self, other, *args, **kwargs)`

Perform element-wise ‘equal’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.equal`.

#### Returns

Result of element-wise ‘equal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.equal`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.equal(other)
```

### 3.1.5.23 `bandhic.band_hic_matrix.exp`

`band_hic_matrix.exp(self, *args, **kwargs)`

Perform element-wise ‘exp’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.exp`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.exp`.

#### Returns

Result of element-wise ‘exp’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.exp`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.exp()
```

### 3.1.5.24 bandhic.band\_hic\_matrix.exp2

`band_hic_matrix.exp2(self, *args, **kwargs)`

Perform element-wise ‘exp2’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.exp2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.exp2`.

#### Returns

Result of element-wise ‘exp2’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.exp2`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.exp2()
```

### 3.1.5.25 bandhic.band\_hic\_matrix.expm1

`band_hic_matrix.expm1(self, *args, **kwargs)`

Perform element-wise ‘expm1’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.expm1`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.expm1`.

#### Returns

Result of element-wise ‘expm1’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.expm1`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.expm1()
```

### 3.1.5.26 bandhic.band\_hic\_matrix.fabs

`band_hic_matrix.fabs(self, *args, **kwargs)`

Perform element-wise ‘fabs’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.fabs`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.fabs`.

#### Returns

Result of element-wise ‘fabs’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.fabs`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.fabs()
```

### 3.1.5.27 bandhic.band\_hic\_matrix.float\_power

`band_hic_matrix.float_power(self, other, *args, **kwargs)`

Perform element-wise ‘float\_power’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.float_power`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.float_power`.

#### Returns

Result of element-wise ‘float\_power’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.float_power`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.float_power(other)
```

### 3.1.5.28 bandhic.band\_hic\_matrix.floor\_divide

`band_hic_matrix.floor_divide(self, other, *args, **kwargs)`

Perform element-wise ‘floor\_divide’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.floor_divide`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.floor_divide`.

#### Returns

Result of element-wise ‘floor\_divide’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.floor_divide`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.floor_divide(other)
```

### 3.1.5.29 bandhic.band\_hic\_matrix.fmod

`band_hic_matrix.fmod(self, other, *args, **kwargs)`

Perform element-wise ‘fmod’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.fmod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.fmod`.

#### Returns

Result of element-wise ‘fmod’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.fmod`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.fmod(other)
```

**3.1.5.30 bandhic.band\_hic\_matrix.gcd**

`band_hic_matrix.gcd(self, other, *args, **kwargs)`

Perform element-wise ‘gcd’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix or array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.gcd`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.gcd`.

**Returns**

Result of element-wise ‘gcd’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.gcd`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.gcd(other)
```

---

### 3.1.5.31 `bandhic.band_hic_matrix.greater`

`band_hic_matrix.greater(self, other, *args, **kwargs)`

Perform element-wise ‘greater’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.greater`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.greater`.

#### Returns

Result of element-wise ‘greater’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.greater`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.greater(other)
```

### 3.1.5.32 `bandhic.band_hic_matrix.greater_equal`

`band_hic_matrix.greater_equal(self, other, *args, **kwargs)`

Perform element-wise ‘greater\_equal’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.greater_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.greater_equal`.

#### Returns

Result of element-wise ‘greater\_equal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.greater_equal`



---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.greater_equal(other)
```

### 3.1.5.33 bandhic.band\_hic\_matrix.heaviside

`band_hic_matrix.heaviside(self, other, *args, **kwargs)`

Perform element-wise ‘heaviside’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.heaviside`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.heaviside`.

#### Returns

Result of element-wise ‘heaviside’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.heaviside`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.heaviside(other)
```

### 3.1.5.34 bandhic.band\_hic\_matrix.hypot

`band_hic_matrix.hypot(self, other, *args, **kwargs)`

Perform element-wise ‘hypot’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.hypot`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.hypot`.

#### Returns

Result of element-wise ‘hypot’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.hypot`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.hypot(other)
```

**3.1.5.35 bandhic.band\_hic\_matrix.invert**

`band_hic_matrix.invert(self, *args, **kwargs)`

Perform element-wise ‘invert’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.invert`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.invert`.

**Returns**

Result of element-wise ‘invert’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.invert`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.invert()
```

**3.1.5.36 bandhic.band\_hic\_matrix.lcm**

`band_hic_matrix.lcm(self, other, *args, **kwargs)`

Perform element-wise ‘lcm’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.lcm`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.lcm`.

---

**Returns**

Result of element-wise ‘lcm’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.lcm`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.lcm(other)
```

### 3.1.5.37 bandhic.band\_hic\_matrix.left\_shift

`band_hic_matrix.left_shift(self, other, *args, **kwargs)`

Perform element-wise ‘left\_shift’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.left_shift`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.left_shift`.

**Returns**

Result of element-wise ‘left\_shift’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.left_shift`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.left_shift(other)
```

---

### 3.1.5.38 bandhic.band\_hic\_matrix.less

`band_hic_matrix.less(self, other, *args, **kwargs)`

Perform element-wise ‘less’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.less`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.less`.

#### Returns

Result of element-wise ‘less’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.less`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.less(other)
```

### 3.1.5.39 bandhic.band\_hic\_matrix.less\_equal

`band_hic_matrix.less_equal(self, other, *args, **kwargs)`

Perform element-wise ‘less\_equal’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.less_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.less_equal`.

#### Returns

Result of element-wise ‘less\_equal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.less_equal`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.less_equal(other)
```

### 3.1.5.40 bandhic.band\_hic\_matrix.log

`band_hic_matrix.log(self, *args, **kwargs)`

Perform element-wise ‘log’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log`.

#### Returns

Result of element-wise ‘log’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log()
```

### 3.1.5.41 bandhic.band\_hic\_matrix.log1p

`band_hic_matrix.log1p(self, *args, **kwargs)`

Perform element-wise ‘log1p’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log1p`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log1p`.

#### Returns

Result of element-wise ‘log1p’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log1p`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.loglp()
```

### 3.1.5.42 bandhic.band\_hic\_matrix.log2

`band_hic_matrix.log2(self, *args, **kwargs)`

Perform element-wise ‘log2’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log2`.

#### Returns

Result of element-wise ‘log2’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log2`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log2()
```

### 3.1.5.43 bandhic.band\_hic\_matrix.log10

`band_hic_matrix.log10(self, *args, **kwargs)`

Perform element-wise ‘log10’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log10`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log10`.

#### Returns

Result of element-wise ‘log10’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log10`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log10()
```

### 3.1.5.44 bandhic.band\_hic\_matrix.logaddexp

`band_hic_matrix.logaddexp(self, other, *args, **kwargs)`

Perform element-wise ‘logaddexp’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logaddexp`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logaddexp`.

#### Returns

Result of element-wise ‘logaddexp’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.logaddexp`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logaddexp(other)
```

### 3.1.5.45 bandhic.band\_hic\_matrix.logaddexp2

`band_hic_matrix.logaddexp2(self, other, *args, **kwargs)`

Perform element-wise ‘logaddexp2’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logaddexp2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logaddexp2`.

#### Returns

Result of element-wise ‘logaddexp2’ operation.

#### Return type

*band\_hic\_matrix*

---

See also:

`numpy.logaddexp2`

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logaddexp2(other)
```

#### 3.1.5.46 `bandhic.band_hic_matrix.logical_and`

`band_hic_matrix.logical_and(self, other, *args, **kwargs)`

Perform element-wise ‘logical\_and’ operation with two inputs.

##### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_and`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_and`.

##### Returns

Result of element-wise ‘logical\_and’ operation.

##### Return type

*band\_hic\_matrix*

See also:

`numpy.logical_and`

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_and(other)
```

#### 3.1.5.47 `bandhic.band_hic_matrix.logical_or`

`band_hic_matrix.logical_or(self, other, *args, **kwargs)`

Perform element-wise ‘logical\_or’ operation with two inputs.

##### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_or`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_or`.



---

**Returns**

Result of element-wise ‘logical\_or’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.logical_or`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_or(other)
```

### 3.1.5.48 bandhic.band\_hic\_matrix.logical\_xor

`band_hic_matrix.logical_xor(self, other, *args, **kwargs)`

Perform element-wise ‘logical\_xor’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_xor`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_xor`.

**Returns**

Result of element-wise ‘logical\_xor’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.logical_xor`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_xor(other)
```

---

### 3.1.5.49 bandhic.band\_hic\_matrix.maximum

`band_hic_matrix.maximum(self, other, *args, **kwargs)`

Perform element-wise ‘maximum’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.maximum`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.maximum`.

#### Returns

Result of element-wise ‘maximum’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.maximum`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.maximum(other)
```

### 3.1.5.50 bandhic.band\_hic\_matrix.minimum

`band_hic_matrix.minimum(self, other, *args, **kwargs)`

Perform element-wise ‘minimum’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.minimum`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.minimum`.

#### Returns

Result of element-wise ‘minimum’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.minimum`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.minimum(other)
```

### 3.1.5.51 bandhic.band\_hic\_matrix.mod

`band_hic_matrix.mod(self, other, *args, **kwargs)`

Perform element-wise ‘mod’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.mod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.mod`.

#### Returns

Result of element-wise ‘mod’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.mod`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.mod(other)
```

### 3.1.5.52 bandhic.band\_hic\_matrix.multiply

`band_hic_matrix.multiply(self, other, *args, **kwargs)`

Perform element-wise ‘multiply’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.multiply`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.multiply`.

#### Returns

Result of element-wise ‘multiply’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.multiply`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.multiply(other)
```

**3.1.5.53 bandhic.band\_hic\_matrix.negative**

`band_hic_matrix.negative(self, *args, **kwargs)`

Perform element-wise ‘negative’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.negative`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.negative`.

**Returns**

Result of element-wise ‘negative’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.negative`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.negative()
```

**3.1.5.54 bandhic.band\_hic\_matrix.not\_equal**

`band_hic_matrix.not_equal(self, other, *args, **kwargs)`

Perform element-wise ‘not\_equal’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.not_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.not_equal`.

---

**Returns**

Result of element-wise ‘not\_equal’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.not_equal`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.not_equal(other)
```

**3.1.5.55 bandhic.band\_hic\_matrix.positive**

`band_hic_matrix.positive(self, *args, **kwargs)`

Perform element-wise ‘positive’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.positive`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.positive`.

**Returns**

Result of element-wise ‘positive’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.positive`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.positive()
```

---

### 3.1.5.56 `bandhic.band_hic_matrix.power`

`band_hic_matrix.power(self, other, *args, **kwargs)`

Perform element-wise ‘power’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.power`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.power`.

#### Returns

Result of element-wise ‘power’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.power`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.power(other)
```

### 3.1.5.57 `bandhic.band_hic_matrix.rad2deg`

`band_hic_matrix.rad2deg(self, *args, **kwargs)`

Perform element-wise ‘rad2deg’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.rad2deg`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.rad2deg`.

#### Returns

Result of element-wise ‘rad2deg’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.rad2deg`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.rad2deg()
```

### 3.1.5.58 bandhic.band\_hic\_matrix.radians

`band_hic_matrix.radians(self, *args, **kwargs)`

Perform element-wise ‘radians’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.radians`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.radians`.

#### Returns

Result of element-wise ‘radians’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.radians`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.radians()
```

### 3.1.5.59 bandhic.band\_hic\_matrix.reciprocal

`band_hic_matrix.reciprocal(self, *args, **kwargs)`

Perform element-wise ‘reciprocal’ operation.

#### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.reciprocal`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.reciprocal`.

#### Returns

Result of element-wise ‘reciprocal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.reciprocal`

---

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.reciprocal()
```

### 3.1.5.60 bandhic.band\_hic\_matrix.remainder

`band_hic_matrix.remainder(self, other, *args, **kwargs)`

Perform element-wise ‘remainder’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.remainder`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.remainder`.

#### Returns

Result of element-wise ‘remainder’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.remainder`

## Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.remainder(other)
```

### 3.1.5.61 bandhic.band\_hic\_matrix.right\_shift

`band_hic_matrix.right_shift(self, other, *args, **kwargs)`

Perform element-wise ‘right\_shift’ operation with two inputs.

#### Parameters

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.right_shift`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.right_shift`.

#### Returns

Result of element-wise ‘right\_shift’ operation.

#### Return type

*band\_hic\_matrix*



---

See also:

`numpy.right_shift`

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.right_shift(other)
```

#### 3.1.5.62 `bandhic.band_hic_matrix.rint`

`band_hic_matrix.rint(self, *args, **kwargs)`

Perform element-wise ‘rint’ operation.

##### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.rint`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.rint`.

##### Returns

Result of element-wise ‘rint’ operation.

##### Return type

*band\_hic\_matrix*

See also:

`numpy.rint`

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.rint()
```

#### 3.1.5.63 `bandhic.band_hic_matrix.sign`

`band_hic_matrix.sign(self, *args, **kwargs)`

Perform element-wise ‘sign’ operation.

##### Parameters

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sign`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sign`.

##### Returns

Result of element-wise ‘sign’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.sign`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sign()
```

**3.1.5.64 bandhic.band\_hic\_matrix.sin**

`band_hic_matrix.sin(self, *args, **kwargs)`

Perform element-wise ‘sin’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sin`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sin`.

**Returns**

Result of element-wise ‘sin’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.sin`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sin()
```

**3.1.5.65 bandhic.band\_hic\_matrix.sinh**

`band_hic_matrix.sinh(self, *args, **kwargs)`

Perform element-wise ‘sinh’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sinh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sinh`.

**Returns**

Result of element-wise ‘sinh’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.sinh`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sinh()
```

### 3.1.5.66 bandhic.band\_hic\_matrix.sqrt

`band_hic_matrix.sqrt(self, *args, **kwargs)`

Perform element-wise ‘sqrt’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sqrt`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sqrt`.

**Returns**

Result of element-wise ‘sqrt’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.sqrt`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sqrt()
```

### 3.1.5.67 bandhic.band\_hic\_matrix.square

`band_hic_matrix.square(self, *args, **kwargs)`

Perform element-wise ‘square’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.square`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.square`.

**Returns**

Result of element-wise ‘square’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.square`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.square()
```

**3.1.5.68 bandhic.band\_hic\_matrix.subtract**

`band_hic_matrix.subtract(self, other, *args, **kwargs)`

Perform element-wise ‘subtract’ operation with two inputs.

**Parameters**

- **self** (*band\_hic\_matrix*) – First input matrix.
- **other** (*band\_hic\_matrix* or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.subtract`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.subtract`.

**Returns**

Result of element-wise ‘subtract’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.subtract`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.subtract(other)
```

**3.1.5.69 bandhic.band\_hic\_matrix.tan**

`band_hic_matrix.tan(self, *args, **kwargs)`

Perform element-wise ‘tan’ operation.

**Parameters**

- **self** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.tan`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.tan`.

---

**Returns**

Result of element-wise ‘tan’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.tan`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.tan()
```

### 3.1.5.70 `bandhic.band_hic_matrix.tanh`

`band_hic_matrix.tanh(self, *args, **kwargs)`

Perform element-wise ‘tanh’ operation.

**Parameters**

- **self** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.tanh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.tanh`.

**Returns**

Result of element-wise ‘tanh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.tanh`

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.tanh()
```

### 3.1.5.71 `bandhic.band_hic_matrix.true_divide`

`band_hic_matrix.true_divide(self, other, *args, **kwargs)`

Perform element-wise ‘true\_divide’ operation with two inputs.

**Parameters**

- **self** (`band_hic_matrix`) – First input matrix.
- **other** (`band_hic_matrix` or *array-like*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.true_divide`.

- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.true_divide`.

#### Returns

Result of element-wise ‘true\_divide’ operation.

#### Return type

*band\_hic\_matrix*

#### See also:

`numpy.true_divide`

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.true_divide(other)
```

### 3.1.6 Data Type Conversion and Representation

This section includes methods for converting the matrix to different data types or representations, such as dense, sparse, or masked arrays.

|                                                    |                                                                            |
|----------------------------------------------------|----------------------------------------------------------------------------|
| <code>band_hic_matrix.todense()</code>             | Convert the band matrix to a dense format.                                 |
| <code>band_hic_matrix.tocoo()</code>               | Convert the matrix to COO format.                                          |
| <code>band_hic_matrix.tocsr()</code>               | Convert the matrix to CSR format.                                          |
| <code>band_hic_matrix.copy()</code>                | Deep copy the object.                                                      |
| <code>band_hic_matrix.astype(dtype[, copy])</code> | Cast data to new dtype.                                                    |
| <code>band_hic_matrix.__repr__()</code>            | Return a string representation of the <code>band_hic_matrix</code> object. |
| <code>band_hic_matrix.__str__()</code>             | Return a string representation of the <code>band_hic_matrix</code> object. |
| <code>band_hic_matrix.__array__([copy])</code>     | Return the data as a NumPy array.                                          |
| <code>band_hic_matrix.__array_priority__()</code>  | Return the priority for array operations.                                  |

#### 3.1.6.1 `bandhic.band_hic_matrix.todense`

`band_hic_matrix.todense()`

Convert the band matrix to a dense format.

#### Returns

The dense (square) matrix. Masked if mask is set.

#### Return type

`ndarray` or `MaskedArray`

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> dense = mat.todense()
>>> dense.shape
(3, 3)
```

### 3.1.6.2 bandhic.band\_hic\_matrix.tocoo

`band_hic_matrix.tocoo()`

Convert the matrix to COO format.

**Returns**

The matrix in scipy COO sparse format.

**Return type**

coo\_array

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> coo = mat.tocoo()
>>> coo.shape
(3, 3)
```

### 3.1.6.3 bandhic.band\_hic\_matrix.tocsr

`band_hic_matrix.tocsr()`

Convert the matrix to CSR format.

**Returns**

The matrix in scipy CSR sparse format.

**Return type**

csr\_array

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> csr = mat.tocsr()
>>> csr.shape
(3, 3)
```

---

### 3.1.6.4 bandhic.band\_hic\_matrix.copy

`band_hic_matrix.copy()`

Deep copy the object.

**Returns**

A deep copy.

**Return type**

*band\_hic\_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat2 = mat.copy()
```

### 3.1.6.5 bandhic.band\_hic\_matrix.astype

`band_hic_matrix.astype(dtype, copy=False)`

Cast data to new dtype.

**Parameters**

- **type** (*data-type*) – Target dtype.
- **copy** (*bool*) – If True, the operation is performed in place. Default is False.
- **dtype** (*type*) –

**Return type**

*band\_hic\_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat = mat.astype(np.float32)
```

**Return type**

*band\_hic\_matrix*

**Parameters**

- **dtype** (*type*) –
- **copy** (*bool*) –



---

### 3.1.6.6 bandhic.band\_hic\_matrix.\_\_repr\_\_

band\_hic\_matrix.\_\_repr\_\_()

Return a string representation of the band\_hic\_matrix object.

**Returns**

A string representation of the object.

**Return type**

str

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> repr(mat)
"band_hic_matrix(shape=(3, 3), diag_num=2, dtype=<class 'numpy.float64'>)"
```

### 3.1.6.7 bandhic.band\_hic\_matrix.\_\_str\_\_

band\_hic\_matrix.\_\_str\_\_()

Return a string representation of the band\_hic\_matrix object.

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> print(mat)
band_hic_matrix(shape=(3, 3), diag_num=2, dtype=<class 'numpy.float64'>)
```

### 3.1.6.8 bandhic.band\_hic\_matrix.\_\_array\_\_

band\_hic\_matrix.\_\_array\_\_(copy=False)

Return the data as a NumPy array.

**Parameters**

**copy** (bool, optional) – If True, returns a copy. Default is False.

**Returns**

Underlying band data array.

**Return type**

ndarray

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> arr = np.array(mat)
```

### 3.1.6.9 bandhic.band\_hic\_matrix.\_\_array\_priority\_\_

`band_hic_matrix.__array_priority__()`

Return the priority for array operations.

**Returns**

Priority value.

**Return type**

int

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.__array_priority__()
100
```

## 3.1.7 Other Methods

This section includes additional methods that do not fall into the categories above but provide other useful operations for *band\_hic\_matrix*.

|                                             |                                                                        |
|---------------------------------------------|------------------------------------------------------------------------|
| <code>band_hic_matrix.memory_usage()</code> | Compute memory usage of <i>band_hic_matrix</i> object.                 |
| <code>band_hic_matrix.__len__()</code>      | Return the number of rows in the <i>band_hic_matrix</i> object.        |
| <code>band_hic_matrix.__hash__()</code>     | Return a hash value for the <i>band_hic_matrix</i> object.             |
| <code>band_hic_matrix.__bool__()</code>     | Truth value of the <i>band_hic_matrix</i> , following NumPy semantics. |
| <code>band_hic_matrix.dump(filename)</code> | Save the <i>band_hic_matrix</i> object to a file.                      |

### 3.1.7.1 bandhic.band\_hic\_matrix.memory\_usage

`band_hic_matrix.memory_usage()`

Compute memory usage of *band\_hic\_matrix* object.

**Returns**

Size in bytes.

**Return type**

int

---

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.memory_usage()
772
```

### 3.1.7.2 bandhic.band\_hic\_matrix.\_\_len\_\_

`band_hic_matrix.__len__()`

Return the number of rows in the `band_hic_matrix` object.

**Returns**

Number of rows.

**Return type**

int

## Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> len(mat)
3
```

### 3.1.7.3 bandhic.band\_hic\_matrix.\_\_hash\_\_

`band_hic_matrix.__hash__()`

Return a hash value for the `band_hic_matrix` object.

**Returns**

Hash value.

**Return type**

int

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> hashed=hash(mat)
```

---

#### 3.1.7.4 bandhic.band\_hic\_matrix.\_\_bool\_\_

`band_hic_matrix.__bool__()`

Truth value of the `band_hic_matrix`, following NumPy semantics.

**Returns**

If the matrix has exactly one element (shape (1,1)), returns its truth value; otherwise raises a `ValueError`.

**Return type**

`bool`

**Raises**

**ValueError** – If the matrix contains more than one element.

#### 3.1.7.5 bandhic.band\_hic\_matrix.dump

`band_hic_matrix.dump(filename)`

Save the `band_hic_matrix` object to a file.

**Parameters**

**filename** (*str*) – The name of the file to save the object to.

**Return type**

`None`

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.dump('myfile.npz')
```

**Return type**

`None`

**Parameters**

**filename** (*str*) –

## 3.2 Creation Functions

These functions create BandHiC matrices with specific properties.

---

|                                                               |                                                                                                      |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>bandhic.ones(shape[, diag_num, dtype])</code>           | Create a <code>band_hic_matrix</code> object filled with ones.                                       |
| <code>bandhic.zeros(shape[, diag_num, dtype])</code>          | Create a <code>band_hic_matrix</code> object filled with zeros.                                      |
| <code>bandhic.eye(shape[, diag_num, dtype])</code>            | Create a <code>band_hic_matrix</code> object filled as an identity matrix.                           |
| <code>bandhic.full(shape, fill_value[, diag_num, ...])</code> | Create a <code>band_hic_matrix</code> object filled with a specified value.                          |
| <code>bandhic.ones_like(other[, dtype])</code>                | Create a <code>band_hic_matrix</code> object matching another matrix, filled with ones.              |
| <code>bandhic.zeros_like(other[, dtype])</code>               | Create a <code>band_hic_matrix</code> object matching another matrix, filled with zeros.             |
| <code>bandhic.eye_like(other[, dtype])</code>                 | Create a <code>band_hic_matrix</code> object matching another matrix, filled as an identity matrix.  |
| <code>bandhic.full_like(other, fill_value[, dtype])</code>    | Create a <code>band_hic_matrix</code> object matching another matrix, filled with a specified value. |

---

### 3.2.1 bandhic.ones

`bandhic.ones(shape, diag_num=1, dtype=<class 'numpy.float64'>)`

Create a `band_hic_matrix` object filled with ones.

#### Parameters

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag\_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is `np.float64`.

#### Returns

A `band_hic_matrix` object with all entries filled with ones.

#### Return type

*band\_hic\_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.2 bandhic.zeros

`bandhic.zeros(shape, diag_num=1, dtype=<class 'numpy.float64'>)`

Create a `band_hic_matrix` object filled with zeros.

#### Parameters

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag\_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is `np.float64`.

---

### Returns

A `band_hic_matrix` object with all entries filled with zeros.

### Return type

*band\_hic\_matrix*

### Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

## 3.2.3 bandhic.eye

`bandhic.eye(shape, diag_num=1, dtype=<class 'numpy.float64'>)`

Create a `band_hic_matrix` object filled as an identity matrix.

### Parameters

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag\_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is `np.float64`.

### Returns

A `band_hic_matrix` object with ones on the main diagonal and zeros elsewhere.

### Return type

*band\_hic\_matrix*

### Examples

```
>>> mat = eye((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

## 3.2.4 bandhic.full

`bandhic.full(shape, fill_value, diag_num=1, dtype=<class 'numpy.float64'>)`

Create a `band_hic_matrix` object filled with a specified value.

### Parameters

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **fill\_value** (*scalar*) – Value to fill the matrix with.
- **diag\_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is `np.float64`.

### Returns

A `band_hic_matrix` object with all entries filled with *fill\_value*.

---

**Return type**

*band\_hic\_matrix*

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.full((5, 5), fill_value=7, diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.5 bandhic.ones\_like

`bandhic.ones_like(other, dtype=None)`

Create a `band_hic_matrix` object matching another matrix, filled with ones.

**Parameters**

**other** (*band\_hic\_matrix*) – Reference matrix.

**Returns**

A `band_hic_matrix` object matching *other*, filled with ones.

**Return type**

*band\_hic\_matrix*

**Examples**

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = ones_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.6 bandhic.zeros\_like

`bandhic.zeros_like(other, dtype=None)`

Create a `band_hic_matrix` object matching another matrix, filled with zeros.

**Parameters**

**other** (*band\_hic\_matrix*) – Reference matrix.

**Returns**

A `band_hic_matrix` object matching *other*, filled with zeros.

**Return type**

*band\_hic\_matrix*

---

## Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = zeros_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.7 bandhic.eye\_like

`bandhic.eye_like(other, dtype=None)`

Create a `band_hic_matrix` object matching another matrix, filled as an identity matrix.

**Parameters**

**other** (`band_hic_matrix`) – Reference matrix.

**Returns**

A `band_hic_matrix` object matching *other*, filled as an identity matrix.

**Return type**

*band\_hic\_matrix*

## Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = eye_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.8 bandhic.full\_like

`bandhic.full_like(other, fill_value, dtype=None)`

Create a `band_hic_matrix` object matching another matrix, filled with a specified value.

**Parameters**

- **other** (`band_hic_matrix`) – Reference matrix.
- **fill\_value** (*scalar*) – Value to fill the matrix.

**Returns**

A `band_hic_matrix` object matching *other*, filled with *fill\_value*.

**Return type**

*band\_hic\_matrix*



---

## Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = full_like(mat_ref, fill_value=9)
>>> isinstance(mat, band_hic_matrix)
True
```

## 3.3 Input/Output Functions

These functions handle input and output operations for BandHiC matrices, including saving and loading from various formats.

|                                                                |                                                                                                                                                           |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>bandhic.save_npz(file_name, mat)</code>                  | Save a <code>band_hic_matrix</code> to a <code>.npz</code> file.                                                                                          |
| <code>bandhic.load_npz(file_name)</code>                       | Load a <code>band_hic_matrix</code> from a <code>.npz</code> file.                                                                                        |
| <code>bandhic.straw_chr(hic_file, chrom, ...[, ...])</code>    | Read Hi-C data from a <code>.hic</code> file and return a <code>band_hic_matrix</code> .                                                                  |
| <code>bandhic.straw_all_chrs(hic_file, resolution, ...)</code> | Read Hi-C data from a <code>.hic</code> file for all chromosomes and return a dictionary of <code>band_hic_matrix</code> objects.                         |
| <code>bandhic.cooler_chr(file_path, chrom, diag_num)</code>    | Read Hi-C data from a <code>.cool</code> or <code>.mcool</code> file and return a <code>band_hic_matrix</code> .                                          |
| <code>bandhic.cooler_all_chrs(file_path, diag_num)</code>      | Read Hi-C data from a <code>.cool</code> or <code>.mcool</code> file for all chromosomes and return a dictionary of <code>band_hic_matrix</code> objects. |

### 3.3.1 bandhic.save\_npz

`bandhic.save_npz(file_name, mat)`

Save a `band_hic_matrix` to a `.npz` file.

#### Parameters

- **file\_name** (*str*) – Path to save the `.npz` file.
- **mat** (`band_hic_matrix`) – The `band_hic_matrix` object to save.

#### Return type

None

## Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(5), diag_num=3)
>>> save_npz('./test/sample.npz', mat)
```

#### Return type

None

#### Parameters

- **file\_name** (*str*) –
- **mat** (`band_hic_matrix`) –

---

### 3.3.2 bandhic.load\_npz

`bandhic.load_npz(file_name)`

Load a `band_hic_matrix` from a `.npz` file.

**Parameters**

**file\_name** (*str*) – Path to the `.npz` file.

**Returns**

A `band_hic_matrix` object loaded from the file.

**Return type**

*band\_hic\_matrix*

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.load_npz('./test/sample.npz')
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.3.3 bandhic.straw\_chr

`bandhic.straw_chr(hic_file, chrom, resolution, diag_num, data_type='observed', normalization='NONE', unit='BP')`

Read Hi-C data from a `.hic` file and return a `band_hic_matrix`.

**Parameters**

- **hic\_file** (*str*) – Path to the `.hic` file. This file should be in the Hi-C format compatible with `hicstraw`. Local or remote paths are supported.
- **chrom** (*str*) – Chromosome name (e.g., `'chr1'`, `'chrX'`). Short names like `'1'`, `'X'` are also accepted.
- **resolution** (*int*) – Resolution of the Hi-C data. Such as 10000 for 10kb resolution.
- **diag\_num** (*int*) – Number of diagonals to consider.
- **data\_type** (*str*, *optional*) – Type of data to read from the Hi-C file. Default is `'observed'`. Other options include `'expected'`, `'balanced'`, etc. See *hicstraw* documentation for more details.
- **normalization** (*str*, *optional*) – Normalization method to apply. Default is `'NONE'`. Other options include `'VC'`, `'VC_SQRT'`, `'KR'`, `'SCALE'`, etc. See *hicstraw* documentation for more details.
- **unit** (*str*, *optional*) – Unit of measurement for the Hi-C data. Default is `'BP'` (base pairs). Other options include `'FRAG'` (fragments), etc.

**Return type**

*band\_hic\_matrix*

**See also:**

[hicstraw documentation](#) Python interface to read Hi-C data files using *.hic* format.

---

**Returns**

A `band_hic_matrix` object containing the Hi-C data.

**Return type**

*band\_hic\_matrix*

**Raises**

**ValueError** – If the file cannot be parsed or parameters are invalid.

**Parameters**

- **hic\_file** (*str*) –
- **chrom** (*str*) –
- **resolution** (*int*) –
- **diag\_num** (*int*) –
- **data\_type** (*str*) –
- **normalization** (*str*) –
- **unit** (*str*) –

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.straw_chr('./data/GSE130275_mESC_WT_combined_1.3B_microc.hic', 'chr1',
↳ resolution=10000, diag_num=200)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.3.4 bandhic.straw\_all\_chrs

`bandhic.straw_all_chrs(hic_file, resolution, diag_num, data_type='observed', normalization='NONE', unit='BP')`

Read Hi-C data from a .hic file for all chromosomes and return a dictionary of `band_hic_matrix` objects.

**Parameters**

- **hic\_file** (*str*) – Path to the .hic file. This file should be in the Hi-C format compatible with hicstraw. Local or remote paths are supported.
- **resolution** (*int*) – Resolution of the Hi-C data. Such as 10000 for 10kb resolution.
- **diag\_num** (*int*) – Number of diagonals to consider.
- **data\_type** (*str*, *optional*) – Type of data to read from the Hi-C file. Default is 'observed'. Other options include 'expected', 'balanced', etc. See *hicstraw* documentation for more details.
- **normalization** (*str*, *optional*) – Normalization method to apply. Default is 'NONE'. Other options include 'VC', 'VC\_SQRT', 'KR', 'SCALE', etc. See *hicstraw* documentation for more details.
- **unit** (*str*, *optional*) – Unit of measurement for the Hi-C data. Default is 'BP' (base pairs). Other options include 'FRAG' (fragments), etc.

#### Returns

A dictionary mapping chromosome names to `band_hic_matrix` objects containing the Hi-C data.

#### Return type

`Dict[str, band_hic_matrix]`

#### Raises

**ValueError** – If the file cannot be parsed or parameters are invalid.

### Examples

```
>>> import bandhic as bh
>>> mats = bh.straw_all_chrs('./data/GSE130275_mESC_WT_combined_1.3B_microc.hic',
↪ resolution=10000, diag_num=200)
>>> isinstance(mats['chr1'], band_hic_matrix)
True
```

### 3.3.5 bandhic.cooler\_chr

`bandhic.cooler_chr(file_path, chrom, diag_num, cell_id=None, resolution=None, balance=True)`

Read Hi-C data from a `.cool` or `.mcool` file and return a `band_hic_matrix`.

#### Parameters

- **file\_path** (*str*) – Path to the `.cool`, `.mcool` or `.scool` file.
- **chrom** (*str*) – Chromosome name.
- **diag\_num** (*int*) – Number of diagonals to consider.
- **cell\_id** (*str, optional*) – Cell ID for `.scool` files.
- **resolution** (*int, optional*) – Resolution of the Hi-C data.
- **balance** (*bool, optional*) – If True, use balanced data. Default is False. This parameter is specific to cooler files.

#### Returns

A `band_hic_matrix` object containing the Hi-C data.

#### Return type

*band\_hic\_matrix*

#### Raises

**ValueError** – If the cooler file is invalid or parameters are incorrect.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.cooler_chr('./data/yeast.10kb.cool', 'chrI', resolution=10000, diag_
↪ num=10)
>>> isinstance(mat, band_hic_matrix)
True
```

#### See also:

[cooler documentation](#) Official documentation for the Cooler format and API usage.

---

### 3.3.6 bandhic.cooler\_all\_chrs

`bandhic.cooler_all_chrs(file_path, diag_num, resolution=None, cell_id=None, balance=True)`

Read Hi-C data from a .cool or .mcool file for all chromosomes and return a dictionary of `band_hic_matrix` objects.

#### Parameters

- **file\_path** (*str*) – Path to the .cool, .mcool or .scool file.
- **diag\_num** (*int*) – Number of diagonals to consider.
- **resolution** (*int*, *optional*) – Resolution of the Hi-C data.
- **cell\_id** (*str*, *optional*) – Cell ID for .scool files.
- **balance** (*bool*, *optional*) – If True, use balanced data. Default is False. This parameter is specific to cooler files.

#### Returns

A dictionary mapping chromosome names to `band_hic_matrix` objects containing the Hi-C data.

#### Return type

Dict[str, `band_hic_matrix`]

#### Raises

**ValueError** – If the cooler file is invalid or parameters are incorrect.

#### Examples

```
>>> import bandhic as bh
>>> mats = bh.cooler_all_chrs('/Users/wwb/Documents/workspace/BandHiC-Master/data/
↳ yeast.10kb.cool', diag_num=10, resolution=100000)
>>> isinstance(mats['chrI'], band_hic_matrix)
True
```

## 3.4 Vectorized operations for BandHiC matrices

These functions perform element-wise operations on BandHiC matrices, allowing for efficient computation across the matrix elements. These functions are wrapped around numpy ufuncs to provide a consistent interface for element-wise operations.

|                                                       |                                                           |
|-------------------------------------------------------|-----------------------------------------------------------|
| <code>bandhic.absolute(x, *args, **kwargs)</code>     | Perform element-wise 'absolute' operation.                |
| <code>bandhic.add(x1, x2, *args, **kwargs)</code>     | Perform element-wise 'add' operation with two inputs.     |
| <code>bandhic.arccos(x, *args, **kwargs)</code>       | Perform element-wise 'arccos' operation.                  |
| <code>bandhic.arccosh(x, *args, **kwargs)</code>      | Perform element-wise 'arccosh' operation.                 |
| <code>bandhic.arcsin(x, *args, **kwargs)</code>       | Perform element-wise 'arcsin' operation.                  |
| <code>bandhic.arcsinh(x, *args, **kwargs)</code>      | Perform element-wise 'arcsinh' operation.                 |
| <code>bandhic.arctan(x, *args, **kwargs)</code>       | Perform element-wise 'arctan' operation.                  |
| <code>bandhic.arctan2(x1, x2, *args, **kwargs)</code> | Perform element-wise 'arctan2' operation with two inputs. |
| <code>bandhic.arctanh(x, *args, **kwargs)</code>      | Perform element-wise 'arctanh' operation.                 |

continues on next page

Table 3.2 – continued from previous page

|                                                        |                                                                 |
|--------------------------------------------------------|-----------------------------------------------------------------|
| <i>bandhic.bitwise_and</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'bitwise_and' operation with two inputs.   |
| <i>bandhic.bitwise_or</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'bitwise_or' operation with two inputs.    |
| <i>bandhic.bitwise_xor</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'bitwise_xor' operation with two inputs.   |
| <i>bandhic.cbrt</i> (x, *args, **kwargs)               | Perform element-wise 'cbrt' operation.                          |
| <i>bandhic.conj</i> (x, *args, **kwargs)               | Perform element-wise 'conj' operation.                          |
| <i>bandhic.conjugate</i> (x, *args, **kwargs)          | Perform element-wise 'conjugate' operation.                     |
| <i>bandhic.cos</i> (x, *args, **kwargs)                | Perform element-wise 'cos' operation.                           |
| <i>bandhic.cosh</i> (x, *args, **kwargs)               | Perform element-wise 'cosh' operation.                          |
| <i>bandhic.deg2rad</i> (x, *args, **kwargs)            | Perform element-wise 'deg2rad' operation.                       |
| <i>bandhic.degrees</i> (x, *args, **kwargs)            | Perform element-wise 'degrees' operation.                       |
| <i>bandhic.divide</i> (x1, x2, *args, **kwargs)        | Perform element-wise 'divide' operation with two inputs.        |
| <i>bandhic.divmod</i> (x1, x2, *args, **kwargs)        | Perform element-wise 'divmod' operation with two inputs.        |
| <i>bandhic.equal</i> (x1, x2, *args, **kwargs)         | Perform element-wise 'equal' operation with two inputs.         |
| <i>bandhic.exp</i> (x, *args, **kwargs)                | Perform element-wise 'exp' operation.                           |
| <i>bandhic.exp2</i> (x, *args, **kwargs)               | Perform element-wise 'exp2' operation.                          |
| <i>bandhic.expml</i> (x, *args, **kwargs)              | Perform element-wise 'expml' operation.                         |
| <i>bandhic.fabs</i> (x, *args, **kwargs)               | Perform element-wise 'fabs' operation.                          |
| <i>bandhic.float_power</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'float_power' operation with two inputs.   |
| <i>bandhic.floor_divide</i> (x1, x2, *args, **kwargs)  | Perform element-wise 'floor_divide' operation with two inputs.  |
| <i>bandhic.fmod</i> (x1, x2, *args, **kwargs)          | Perform element-wise 'fmod' operation with two inputs.          |
| <i>bandhic.gcd</i> (x1, x2, *args, **kwargs)           | Perform element-wise 'gcd' operation with two inputs.           |
| <i>bandhic.greater</i> (x1, x2, *args, **kwargs)       | Perform element-wise 'greater' operation with two inputs.       |
| <i>bandhic.greater_equal</i> (x1, x2, *args, **kwargs) | Perform element-wise 'greater_equal' operation with two inputs. |
| <i>bandhic.heaviside</i> (x1, x2, *args, **kwargs)     | Perform element-wise 'heaviside' operation with two inputs.     |
| <i>bandhic.hypot</i> (x1, x2, *args, **kwargs)         | Perform element-wise 'hypot' operation with two inputs.         |
| <i>bandhic.invert</i> (x, *args, **kwargs)             | Perform element-wise 'invert' operation.                        |
| <i>bandhic.lcm</i> (x1, x2, *args, **kwargs)           | Perform element-wise 'lcm' operation with two inputs.           |
| <i>bandhic.left_shift</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'left_shift' operation with two inputs.    |
| <i>bandhic.less</i> (x1, x2, *args, **kwargs)          | Perform element-wise 'less' operation with two inputs.          |
| <i>bandhic.less_equal</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'less_equal' operation with two inputs.    |
| <i>bandhic.log</i> (x, *args, **kwargs)                | Perform element-wise 'log' operation.                           |
| <i>bandhic.log1p</i> (x, *args, **kwargs)              | Perform element-wise 'log1p' operation.                         |
| <i>bandhic.log2</i> (x, *args, **kwargs)               | Perform element-wise 'log2' operation.                          |
| <i>bandhic.log10</i> (x, *args, **kwargs)              | Perform element-wise 'log10' operation.                         |
| <i>bandhic.logaddexp</i> (x1, x2, *args, **kwargs)     | Perform element-wise 'logaddexp' operation with two inputs.     |
| <i>bandhic.logaddexp2</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'logaddexp2' operation with two inputs.    |
| <i>bandhic.logical_and</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'logical_and' operation with two inputs.   |

continues on next page

Table 3.2 – continued from previous page

|                                                      |                                                               |
|------------------------------------------------------|---------------------------------------------------------------|
| <i>bandhic.logical_or</i> (x1, x2, *args, **kwargs)  | Perform element-wise 'logical_or' operation with two inputs.  |
| <i>bandhic.logical_xor</i> (x1, x2, *args, **kwargs) | Perform element-wise 'logical_xor' operation with two inputs. |
| <i>bandhic.maximum</i> (x1, x2, *args, **kwargs)     | Perform element-wise 'maximum' operation with two inputs.     |
| <i>bandhic.minimum</i> (x1, x2, *args, **kwargs)     | Perform element-wise 'minimum' operation with two inputs.     |
| <i>bandhic.mod</i> (x1, x2, *args, **kwargs)         | Perform element-wise 'mod' operation with two inputs.         |
| <i>bandhic.multiply</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'multiply' operation with two inputs.    |
| <i>bandhic.negative</i> (x, *args, **kwargs)         | Perform element-wise 'negative' operation.                    |
| <i>bandhic.not_equal</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'not_equal' operation with two inputs.   |
| <i>bandhic.positive</i> (x, *args, **kwargs)         | Perform element-wise 'positive' operation.                    |
| <i>bandhic.power</i> (x1, x2, *args, **kwargs)       | Perform element-wise 'power' operation with two inputs.       |
| <i>bandhic.rad2deg</i> (x, *args, **kwargs)          | Perform element-wise 'rad2deg' operation.                     |
| <i>bandhic.radians</i> (x, *args, **kwargs)          | Perform element-wise 'radians' operation.                     |
| <i>bandhic.reciprocal</i> (x, *args, **kwargs)       | Perform element-wise 'reciprocal' operation.                  |
| <i>bandhic.remainder</i> (x1, x2, *args, **kwargs)   | Perform element-wise 'remainder' operation with two inputs.   |
| <i>bandhic.right_shift</i> (x1, x2, *args, **kwargs) | Perform element-wise 'right_shift' operation with two inputs. |
| <i>bandhic rint</i> (x, *args, **kwargs)             | Perform element-wise 'rint' operation.                        |
| <i>bandhic.sign</i> (x, *args, **kwargs)             | Perform element-wise 'sign' operation.                        |
| <i>bandhic.sin</i> (x, *args, **kwargs)              | Perform element-wise 'sin' operation.                         |
| <i>bandhic.sinh</i> (x, *args, **kwargs)             | Perform element-wise 'sinh' operation.                        |
| <i>bandhic.sqrt</i> (x, *args, **kwargs)             | Perform element-wise 'sqrt' operation.                        |
| <i>bandhic.square</i> (x, *args, **kwargs)           | Perform element-wise 'square' operation.                      |
| <i>bandhic.subtract</i> (x1, x2, *args, **kwargs)    | Perform element-wise 'subtract' operation with two inputs.    |
| <i>bandhic.tan</i> (x, *args, **kwargs)              | Perform element-wise 'tan' operation.                         |
| <i>bandhic.tanh</i> (x, *args, **kwargs)             | Perform element-wise 'tanh' operation.                        |
| <i>bandhic.true_divide</i> (x1, x2, *args, **kwargs) | Perform element-wise 'true_divide' operation with two inputs. |

### 3.4.1 bandhic.absolute

**bandhic.absolute**(x, \*args, \*\*kwargs)

Perform element-wise ‘absolute’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.absolute`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.absolute`.

#### Returns

Result of element-wise ‘absolute’ operation.

#### Return type

*band\_hic\_matrix*

---

See also:

`numpy.absolute`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.absolute(x1)
```

## 3.4.2 bandhic.add

`bandhic.add(x1, x2, *args, **kwargs)`

Perform element-wise ‘add’ operation with two inputs.

### Parameters

- **x1** (`band_hic_matrix`, `ndarray` or `scalar`) – First input matrix.
- **x2** (`band_hic_matrix`, `ndarray` or `scalar`) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.add`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.add`.

### Returns

Result of element-wise ‘add’ operation.

### Return type

`band_hic_matrix`

See also:

`numpy.add`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.add(x1, x2)
```

## 3.4.3 bandhic.arccos

`bandhic.arccos(x, *args, **kwargs)`

Perform element-wise ‘arccos’ operation.

### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.arccos`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.arccos`.



---

**Returns**

Result of element-wise ‘arccos’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arccos`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arccos(x1)
```

### 3.4.4 bandhic.arccosh

`bandhic.arccosh(x, *args, **kwargs)`

Perform element-wise ‘arccosh’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arccosh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arccosh`.

**Returns**

Result of element-wise ‘arccosh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arccosh`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arccosh(x1)
```

### 3.4.5 bandhic.arcsin

`bandhic.arcsin(x, *args, **kwargs)`

Perform element-wise ‘arcsin’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arcsin`.

- 
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arcsin`.

**Returns**

Result of element-wise ‘arcsin’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arcsin`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arcsin(x1)
```

### 3.4.6 `bandhic.arcsinh`

`bandhic.arcsinh`(*x*, *\*args*, *\*\*kwargs*)

Perform element-wise ‘arcsinh’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arcsinh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arcsinh`.

**Returns**

Result of element-wise ‘arcsinh’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.arcsinh`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arcsinh(x1)
```

---

### 3.4.7 bandhic.arctan

`bandhic.arctan(x, *args, **kwargs)`

Perform element-wise ‘arctan’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arctan`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arctan`.

#### Returns

Result of element-wise ‘arctan’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.arctan`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arctan(x1)
```

### 3.4.8 bandhic.arctan2

`bandhic.arctan2(x1, x2, *args, **kwargs)`

Perform element-wise ‘arctan2’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.arctan2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.arctan2`.

#### Returns

Result of element-wise ‘arctan2’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.arctan2`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.arctan2(x1, x2)
```

### 3.4.9 bandhic.arctanh

`bandhic.arctanh(x, *args, **kwargs)`

Perform element-wise ‘arctanh’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.arctanh`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.arctanh`.

#### Returns

Result of element-wise ‘arctanh’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.arctanh`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arctanh(x1)
```

### 3.4.10 bandhic.bitwise\_and

`bandhic.bitwise_and(x1, x2, *args, **kwargs)`

Perform element-wise ‘bitwise\_and’ operation with two inputs.

#### Parameters

- **x1** (`band_hic_matrix`, *ndarray or scalar*) – First input matrix.
- **x2** (`band_hic_matrix`, *ndarray or scalar*) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.bitwise_and`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.bitwise_and`.

#### Returns

Result of element-wise ‘bitwise\_and’ operation.

#### Return type

*band\_hic\_matrix*

---

See also:

`numpy.bitwise_and`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_and(x1, x2)
```

## 3.4.11 bandhic.bitwise\_or

`bandhic.bitwise_or(x1, x2, *args, **kwargs)`

Perform element-wise ‘bitwise\_or’ operation with two inputs.

### Parameters

- **x1** (`band_hic_matrix`, `ndarray` or `scalar`) – First input matrix.
- **x2** (`band_hic_matrix`, `ndarray` or `scalar`) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.bitwise_or`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.bitwise_or`.

### Returns

Result of element-wise ‘bitwise\_or’ operation.

### Return type

*band\_hic\_matrix*

See also:

`numpy.bitwise_or`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_or(x1, x2)
```

## 3.4.12 bandhic.bitwise\_xor

`bandhic.bitwise_xor(x1, x2, *args, **kwargs)`

Perform element-wise ‘bitwise\_xor’ operation with two inputs.

### Parameters

- **x1** (`band_hic_matrix`, `ndarray` or `scalar`) – First input matrix.
- **x2** (`band_hic_matrix`, `ndarray` or `scalar`) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.bitwise_xor`.

- 
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.bitwise_xor`.

**Returns**

Result of element-wise ‘bitwise\_xor’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.bitwise_xor`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_xor(x1, x2)
```

### 3.4.13 bandhic.cbrt

`bandhic.cbrt(x, *args, **kwargs)`

Perform element-wise ‘cbrt’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cbrt`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cbrt`.

**Returns**

Result of element-wise ‘cbrt’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.cbrt`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cbrt(x1)
```

---

### 3.4.14 bandhic.conj

`bandhic.conj(x, *args, **kwargs)`

Perform element-wise ‘conj’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.conj`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.conj`.

#### Returns

Result of element-wise ‘conj’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.conj`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.conj(x1)
```

### 3.4.15 bandhic.conjugate

`bandhic.conjugate(x, *args, **kwargs)`

Perform element-wise ‘conjugate’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.conjugate`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.conjugate`.

#### Returns

Result of element-wise ‘conjugate’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.conjugate`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.conjugate(x1)
```

### 3.4.16 bandhic.cos

`bandhic.cos(x, *args, **kwargs)`

Perform element-wise ‘cos’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cos`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cos`.

#### Returns

Result of element-wise ‘cos’ operation.

#### Return type

*band\_hic\_matrix*

See also:

[numpy.cos](#)

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cos(x1)
```

### 3.4.17 bandhic.cosh

`bandhic.cosh(x, *args, **kwargs)`

Perform element-wise ‘cosh’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.cosh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.cosh`.

#### Returns

Result of element-wise ‘cosh’ operation.

#### Return type

*band\_hic\_matrix*

See also:

[numpy.cosh](#)



---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cosh(x1)
```

### 3.4.18 bandhic.deg2rad

`bandhic.deg2rad(x, *args, **kwargs)`

Perform element-wise ‘deg2rad’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.deg2rad`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.deg2rad`.

#### Returns

Result of element-wise ‘deg2rad’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.deg2rad`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.deg2rad(x1)
```

### 3.4.19 bandhic.degrees

`bandhic.degrees(x, *args, **kwargs)`

Perform element-wise ‘degrees’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.degrees`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.degrees`.

#### Returns

Result of element-wise ‘degrees’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.degrees`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.degrees(x1)
```

### 3.4.20 bandhic.divide

`bandhic.divide(x1, x2, *args, **kwargs)`

Perform element-wise ‘divide’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.divide`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.divide`.

#### Returns

Result of element-wise ‘divide’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.divide`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.divide(x1, x2)
```

### 3.4.21 bandhic.divmod

`bandhic.divmod(x1, x2, *args, **kwargs)`

Perform element-wise ‘divmod’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.divmod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.divmod`.

#### Returns

Result of element-wise ‘divmod’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.divmod`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.divmod(x1, x2)
```

### 3.4.22 bandhic.equal

`bandhic.equal(x1, x2, *args, **kwargs)`

Perform element-wise ‘equal’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.equal`.

**Returns**

Result of element-wise ‘equal’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.equal`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.equal(x1, x2)
```

---

### 3.4.23 bandhic.exp

`bandhic.exp(x, *args, **kwargs)`

Perform element-wise ‘exp’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.exp`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.exp`.

#### Returns

Result of element-wise ‘exp’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.exp`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.exp(x1)
```

### 3.4.24 bandhic.exp2

`bandhic.exp2(x, *args, **kwargs)`

Perform element-wise ‘exp2’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.exp2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.exp2`.

#### Returns

Result of element-wise ‘exp2’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.exp2`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.exp2(x1)
```

### 3.4.25 bandhic.expm1

`bandhic.expm1(x, *args, **kwargs)`

Perform element-wise ‘expm1’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.expm1`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.expm1`.

#### Returns

Result of element-wise ‘expm1’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.expm1`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.expm1(x1)
```

### 3.4.26 bandhic.fabs

`bandhic.fabs(x, *args, **kwargs)`

Perform element-wise ‘fabs’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.fabs`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.fabs`.

#### Returns

Result of element-wise ‘fabs’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.fabs`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.fabs(x1)
```

### 3.4.27 bandhic.float\_power

`bandhic.float_power(x1, x2, *args, **kwargs)`

Perform element-wise ‘float\_power’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.float_power`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.float_power`.

#### Returns

Result of element-wise ‘float\_power’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.float_power`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.float_power(x1, x2)
```

### 3.4.28 bandhic.floor\_divide

`bandhic.floor_divide(x1, x2, *args, **kwargs)`

Perform element-wise ‘floor\_divide’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.floor_divide`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.floor_divide`.

#### Returns

Result of element-wise ‘floor\_divide’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.floor_divide`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.floor_divide(x1, x2)
```

### 3.4.29 bandhic.fmod

`bandhic.fmod(x1, x2, *args, **kwargs)`

Perform element-wise ‘fmod’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.fmod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.fmod`.

**Returns**

Result of element-wise ‘fmod’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.fmod`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.fmod(x1, x2)
```

---

### 3.4.30 bandhic.gcd

`bandhic.gcd(x1, x2, *args, **kwargs)`

Perform element-wise ‘gcd’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.gcd`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.gcd`.

#### Returns

Result of element-wise ‘gcd’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.gcd`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.gcd(x1, x2)
```

### 3.4.31 bandhic.greater

`bandhic.greater(x1, x2, *args, **kwargs)`

Perform element-wise ‘greater’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.greater`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.greater`.

#### Returns

Result of element-wise ‘greater’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.greater`



---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.greater(x1, x2)
```

### 3.4.32 bandhic.greater\_equal

`bandhic.greater_equal(x1, x2, *args, **kwargs)`

Perform element-wise ‘greater\_equal’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.greater_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.greater_equal`.

#### Returns

Result of element-wise ‘greater\_equal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.greater_equal`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.greater_equal(x1, x2)
```

### 3.4.33 bandhic.heaviside

`bandhic.heaviside(x1, x2, *args, **kwargs)`

Perform element-wise ‘heaviside’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.heaviside`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.heaviside`.

#### Returns

Result of element-wise ‘heaviside’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.heaviside`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.heaviside(x1, x2)
```

### 3.4.34 bandhic.hypot

`bandhic.hypot(x1, x2, *args, **kwargs)`

Perform element-wise ‘hypot’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.hypot`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.hypot`.

**Returns**

Result of element-wise ‘hypot’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.hypot`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.hypot(x1, x2)
```

---

### 3.4.35 bandhic.invert

`bandhic.invert(x, *args, **kwargs)`

Perform element-wise ‘invert’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.invert`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.invert`.

#### Returns

Result of element-wise ‘invert’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.invert`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.invert(x1)
```

### 3.4.36 bandhic.lcm

`bandhic.lcm(x1, x2, *args, **kwargs)`

Perform element-wise ‘lcm’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.lcm`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.lcm`.

#### Returns

Result of element-wise ‘lcm’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.lcm`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.lcm(x1, x2)
```

### 3.4.37 bandhic.left\_shift

`bandhic.left_shift(x1, x2, *args, **kwargs)`

Perform element-wise ‘left\_shift’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.left_shift`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.left_shift`.

#### Returns

Result of element-wise ‘left\_shift’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.left_shift`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.left_shift(x1, x2)
```

### 3.4.38 bandhic.less

`bandhic.less(x1, x2, *args, **kwargs)`

Perform element-wise ‘less’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.less`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.less`.

#### Returns

Result of element-wise ‘less’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.less`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.less(x1, x2)
```

### 3.4.39 bandhic.less\_equal

`bandhic.less_equal(x1, x2, *args, **kwargs)`

Perform element-wise ‘less\_equal’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.less_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.less_equal`.

**Returns**

Result of element-wise ‘less\_equal’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.less_equal`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.less_equal(x1, x2)
```

---

### 3.4.40 bandhic.log

`bandhic.log(x, *args, **kwargs)`

Perform element-wise ‘log’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log`.

#### Returns

Result of element-wise ‘log’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log(x1)
```

### 3.4.41 bandhic.log1p

`bandhic.log1p(x, *args, **kwargs)`

Perform element-wise ‘log1p’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log1p`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log1p`.

#### Returns

Result of element-wise ‘log1p’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log1p`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log1p(x1)
```

### 3.4.42 bandhic.log2

`bandhic.log2(x, *args, **kwargs)`

Perform element-wise ‘log2’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log2`.

#### Returns

Result of element-wise ‘log2’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log2`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log2(x1)
```

### 3.4.43 bandhic.log10

`bandhic.log10(x, *args, **kwargs)`

Perform element-wise ‘log10’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.log10`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.log10`.

#### Returns

Result of element-wise ‘log10’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.log10`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log10(x1)
```

### 3.4.44 bandhic.logaddexp

`bandhic.logaddexp(x1, x2, *args, **kwargs)`

Perform element-wise ‘logaddexp’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logaddexp`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logaddexp`.

#### Returns

Result of element-wise ‘logaddexp’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.logaddexp`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logaddexp(x1, x2)
```

### 3.4.45 bandhic.logaddexp2

`bandhic.logaddexp2(x1, x2, *args, **kwargs)`

Perform element-wise ‘logaddexp2’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logaddexp2`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logaddexp2`.

#### Returns

Result of element-wise ‘logaddexp2’ operation.



---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.logaddexp2`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logaddexp2(x1, x2)
```

### 3.4.46 bandhic.logical\_and

`bandhic.logical_and(x1, x2, *args, **kwargs)`

Perform element-wise ‘logical\_and’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_and`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_and`.

**Returns**

Result of element-wise ‘logical\_and’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.logical_and`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_and(x1, x2)
```

---

### 3.4.47 bandhic.logical\_or

`bandhic.logical_or(x1, x2, *args, **kwargs)`

Perform element-wise ‘logical\_or’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_or`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_or`.

#### Returns

Result of element-wise ‘logical\_or’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.logical_or`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_or(x1, x2)
```

### 3.4.48 bandhic.logical\_xor

`bandhic.logical_xor(x1, x2, *args, **kwargs)`

Perform element-wise ‘logical\_xor’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.logical_xor`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.logical_xor`.

#### Returns

Result of element-wise ‘logical\_xor’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.logical_xor`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_xor(x1, x2)
```

### 3.4.49 bandhic.maximum

`bandhic.maximum(x1, x2, *args, **kwargs)`

Perform element-wise ‘maximum’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.maximum`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.maximum`.

#### Returns

Result of element-wise ‘maximum’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.maximum`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.maximum(x1, x2)
```

### 3.4.50 bandhic.minimum

`bandhic.minimum(x1, x2, *args, **kwargs)`

Perform element-wise ‘minimum’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.minimum`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.minimum`.

#### Returns

Result of element-wise ‘minimum’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.minimum`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.minimum(x1, x2)
```

### 3.4.51 bandhic.mod

`bandhic.mod(x1, x2, *args, **kwargs)`

Perform element-wise ‘mod’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.mod`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.mod`.

**Returns**

Result of element-wise ‘mod’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.mod`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.mod(x1, x2)
```

---

### 3.4.52 bandhic.multiply

`bandhic.multiply(x1, x2, *args, **kwargs)`

Perform element-wise ‘multiply’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.multiply`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.multiply`.

#### Returns

Result of element-wise ‘multiply’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.multiply`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.multiply(x1, x2)
```

### 3.4.53 bandhic.negative

`bandhic.negative(x, *args, **kwargs)`

Perform element-wise ‘negative’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.negative`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.negative`.

#### Returns

Result of element-wise ‘negative’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.negative`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.negative(x1)
```

### 3.4.54 bandhic.not\_equal

`bandhic.not_equal(x1, x2, *args, **kwargs)`

Perform element-wise ‘not\_equal’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.not_equal`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.not_equal`.

#### Returns

Result of element-wise ‘not\_equal’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.not_equal`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.not_equal(x1, x2)
```

### 3.4.55 bandhic.positive

`bandhic.positive(x, *args, **kwargs)`

Perform element-wise ‘positive’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.positive`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.positive`.

#### Returns

Result of element-wise ‘positive’ operation.

#### Return type

*band\_hic\_matrix*

---

See also:

`numpy.positive`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.positive(x1)
```

## 3.4.56 bandhic.power

`bandhic.power(x1, x2, *args, **kwargs)`

Perform element-wise ‘power’ operation with two inputs.

### Parameters

- **x1** (`band_hic_matrix`, `ndarray` or `scalar`) – First input matrix.
- **x2** (`band_hic_matrix`, `ndarray` or `scalar`) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.power`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.power`.

### Returns

Result of element-wise ‘power’ operation.

### Return type

*band\_hic\_matrix*

See also:

`numpy.power`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.power(x1, x2)
```

## 3.4.57 bandhic.rad2deg

`bandhic.rad2deg(x, *args, **kwargs)`

Perform element-wise ‘rad2deg’ operation.

### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.rad2deg`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.rad2deg`.

---

**Returns**

Result of element-wise ‘rad2deg’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.rad2deg`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.rad2deg(x1)
```

### 3.4.58 bandhic.radians

`bandhic.radians(x, *args, **kwargs)`

Perform element-wise ‘radians’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.radians`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.radians`.

**Returns**

Result of element-wise ‘radians’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.radians`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.radians(x1)
```

### 3.4.59 bandhic.reciprocal

`bandhic.reciprocal(x, *args, **kwargs)`

Perform element-wise ‘reciprocal’ operation.

**Parameters**

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.reciprocal`.



- 
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.reciprocal`.

**Returns**

Result of element-wise ‘reciprocal’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.reciprocal`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.reciprocal(x1)
```

### 3.4.60 `bandhic.remainer`

`bandhic.remainer(x1, x2, *args, **kwargs)`

Perform element-wise ‘remainder’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix, ndarray or scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix, ndarray or scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.remainder`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.remainder`.

**Returns**

Result of element-wise ‘remainder’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.remainder`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.remainder(x1, x2)
```

---

### 3.4.61 bandhic.right\_shift

`bandhic.right_shift(x1, x2, *args, **kwargs)`

Perform element-wise ‘right\_shift’ operation with two inputs.

#### Parameters

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.right_shift`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.right_shift`.

#### Returns

Result of element-wise ‘right\_shift’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.right_shift`

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.right_shift(x1, x2)
```

### 3.4.62 bandhic.rint

`bandhic.rint(x, *args, **kwargs)`

Perform element-wise ‘rint’ operation.

#### Parameters

- **x** (*band\_hic\_matrix*) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.rint`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.rint`.

#### Returns

Result of element-wise ‘rint’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.rint`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.rint(x1)
```

### 3.4.63 bandhic.sign

`bandhic.sign(x, *args, **kwargs)`

Perform element-wise ‘sign’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sign`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sign`.

#### Returns

Result of element-wise ‘sign’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.sign`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sign(x1)
```

### 3.4.64 bandhic.sin

`bandhic.sin(x, *args, **kwargs)`

Perform element-wise ‘sin’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sin`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sin`.

#### Returns

Result of element-wise ‘sin’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.sin`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sin(x1)
```

### 3.4.65 bandhic.sinh

`bandhic.sinh(x, *args, **kwargs)`

Perform element-wise ‘sinh’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sinh`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sinh`.

#### Returns

Result of element-wise ‘sinh’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.sinh`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sinh(x1)
```

### 3.4.66 bandhic.sqrt

`bandhic.sqrt(x, *args, **kwargs)`

Perform element-wise ‘sqrt’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.sqrt`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.sqrt`.

#### Returns

Result of element-wise ‘sqrt’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.sqrt`

---

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sqrt(x1)
```

### 3.4.67 bandhic.square

`bandhic.square(x, *args, **kwargs)`

Perform element-wise ‘square’ operation.

#### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.square`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.square`.

#### Returns

Result of element-wise ‘square’ operation.

#### Return type

*band\_hic\_matrix*

See also:

`numpy.square`

## Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.square(x1)
```

### 3.4.68 bandhic.subtract

`bandhic.subtract(x1, x2, *args, **kwargs)`

Perform element-wise ‘subtract’ operation with two inputs.

#### Parameters

- **x1** (`band_hic_matrix`, *ndarray or scalar*) – First input matrix.
- **x2** (`band_hic_matrix`, *ndarray or scalar*) – Second input for the operation.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.subtract`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.subtract`.

#### Returns

Result of element-wise ‘subtract’ operation.

#### Return type

*band\_hic\_matrix*

---

See also:

`numpy.subtract`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.subtract(x1, x2)
```

## 3.4.69 bandhic.tan

`bandhic.tan(x, *args, **kwargs)`

Perform element-wise ‘tan’ operation.

### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.tan`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.tan`.

### Returns

Result of element-wise ‘tan’ operation.

### Return type

*band\_hic\_matrix*

See also:

`numpy.tan`

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.tan(x1)
```

## 3.4.70 bandhic.tanh

`bandhic.tanh(x, *args, **kwargs)`

Perform element-wise ‘tanh’ operation.

### Parameters

- **x** (`band_hic_matrix`) – Input matrix.
- **\*args** (`tuple`) – Additional positional arguments for `numpy.tanh`.
- **\*\*kwargs** (`dict`) – Keyword arguments for `numpy.tanh`.

### Returns

Result of element-wise ‘tanh’ operation.

---

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.tanh`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.tanh(x1)
```

### 3.4.71 bandhic.true\_divide

`bandhic.true_divide(x1, x2, *args, **kwargs)`

Perform element-wise ‘true\_divide’ operation with two inputs.

**Parameters**

- **x1** (*band\_hic\_matrix*, *ndarray* or *scalar*) – First input matrix.
- **x2** (*band\_hic\_matrix*, *ndarray* or *scalar*) – Second input for the operation.
- **\*args** (*tuple*) – Additional positional arguments for `numpy.true_divide`.
- **\*\*kwargs** (*dict*) – Keyword arguments for `numpy.true_divide`.

**Returns**

Result of element-wise ‘true\_divide’ operation.

**Return type**

*band\_hic\_matrix*

**See also:**

`numpy.true_divide`

**Examples**

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.true_divide(x1, x2)
```

## 3.5 Data Reduction and Normalization

These functions perform data reduction and normalization on BandHiC matrices, allowing for efficient analysis of Hi-C data. These functions are wrapped around `band_hic_matrix` methods to provide a consistent interface for data reduction and normalization.

---

|                                                 |                                                     |
|-------------------------------------------------|-----------------------------------------------------|
| <code>bandhic.min(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.min</code>       |
| <code>bandhic.max(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.max</code>       |
| <code>bandhic.sum(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.sum</code>       |
| <code>bandhic.mean(*args, **kwargs)</code>      | Wrapped from <code>band_hic_matrix.mean</code>      |
| <code>bandhic.std(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.std</code>       |
| <code>bandhic.var(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.var</code>       |
| <code>bandhic.prod(*args, **kwargs)</code>      | Wrapped from <code>band_hic_matrix.prod</code>      |
| <code>bandhic.ptp(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.ptp</code>       |
| <code>bandhic.all(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.all</code>       |
| <code>bandhic.any(*args, **kwargs)</code>       | Wrapped from <code>band_hic_matrix.any</code>       |
| <code>bandhic.clip(*args, **kwargs)</code>      | Wrapped from <code>band_hic_matrix.clip</code>      |
| <code>bandhic.normalize(*args, **kwargs)</code> | Wrapped from <code>band_hic_matrix.normalize</code> |

### 3.5.1 bandhic.min

`bandhic.min(*args, **kwargs)`

Wrapped from `band_hic_matrix.min`

Refer to the documentation: `bandhic.band_hic_matrix.min()`

### 3.5.2 bandhic.max

`bandhic.max(*args, **kwargs)`

Wrapped from `band_hic_matrix.max`

Refer to the documentation: `bandhic.band_hic_matrix.max()`

### 3.5.3 bandhic.sum

`bandhic.sum(*args, **kwargs)`

Wrapped from `band_hic_matrix.sum`

Refer to the documentation: `bandhic.band_hic_matrix.sum()`

### 3.5.4 bandhic.mean

`bandhic.mean(*args, **kwargs)`

Wrapped from `band_hic_matrix.mean`

Refer to the documentation: `bandhic.band_hic_matrix.mean()`



---

### 3.5.5 bandhic.std

`bandhic.std(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.std*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.std\(\)\*](#)

### 3.5.6 bandhic.var

`bandhic.var(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.var*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.var\(\)\*](#)

### 3.5.7 bandhic.prod

`bandhic.prod(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.prod*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.prod\(\)\*](#)

### 3.5.8 bandhic.ptp

`bandhic.ptp(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.ptp*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.ptp\(\)\*](#)

### 3.5.9 bandhic.all

`bandhic.all(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.all*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.all\(\)\*](#)

### 3.5.10 bandhic.any

`bandhic.any(*args, **kwargs)`

Wrapped from *band\_hic\_matrix.any*

Refer to the documentation: [\*bandhic.band\\_hic\\_matrix.any\(\)\*](#)

---

### 3.5.11 bandhic.clip

`bandhic.clip(*args, **kwargs)`

Wrapped from `band_hic_matrix.clip`

Refer to the documentation: `bandhic.band_hic_matrix.clip()`

### 3.5.12 bandhic.normalize

`bandhic.normalize(*args, **kwargs)`

Wrapped from `band_hic_matrix.normalize`

Refer to the documentation: `bandhic.band_hic_matrix.normalize()`

## 3.6 Other Functions

These functions provide additional utilities for BandHiC matrices, including matrix comparison, bias computation, and TAD calling.

|                                                                |                                                                                                |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>bandhic.matrix_equal(a, b)</code>                        | Check if two <code>band_hic_matrix</code> objects are equal.                                   |
| <code>bandhic.assert_band_matrix_equal(a, b)</code>            | Assert that two <code>band_hic_matrix</code> objects are equal.                                |
| <code>bandhic.compute_bin_bias(hic_coo[, verbose, ...])</code> | Compute bias values for Hi-C contact matrices using the Knight-Ruiz normalization algorithm.   |
| <code>bandhic.call_tad(hic_matrix, resolution, ...)</code>     | Detect TADs from a Hi-C matrix using the TopDom algorithm.                                     |
| <code>bandhic.topdom(hic_matrix, bins, window_size)</code>     | Detect TADs using TopDom algorithm.                                                            |
| <code>bandhic.apa(hic_path, resolution, loops_df)</code>       | Compute Aggregate Peak Analysis (APA) around a set of loop anchors.                            |
| <code>bandhic.hiccups(hic_file, chroms, resolutions)</code>    | Run the HiCCUPS loop-calling algorithm on Hi-C data using the BandHiC data structure.          |
| <code>bandhic.load_norm_vector_from_hic(hic_file, ...)</code>  | Load a normalization vector from a .hic file for a given chromosome and resolution.            |
| <code>bandhic.load_norm_vector_from_cooler(...)</code>         | Load a normalization vector from a .cool or .mcool file for a given chromosome and resolution. |
| <code>bandhic.scc(A, B[, max_k, min_n, method])</code>         | Stratified correlation coefficient (SCC) for Hi-C using BandHiC.                               |

### 3.6.1 bandhic.matrix\_equal

`bandhic.matrix_equal(a, b)`

Check if two `band_hic_matrix` objects are equal.

#### Parameters

- **a** (`band_hic_matrix`) – First `band_hic_matrix` object.
- **b** (`band_hic_matrix`) – Second `band_hic_matrix` object.

#### Returns

True if the matrices are equal, False otherwise.

---

**Return type**  
bool

### 3.6.2 bandhic.assert\_band\_matrix\_equal

`bandhic.assert_band_matrix_equal(a, b)`

Assert that two `band_hic_matrix` objects are equal.

**Parameters**

- **a** (`band_hic_matrix`) – First `band_hic_matrix` object.
- **b** (`band_hic_matrix`) – Second `band_hic_matrix` object.

**Raises**

**AssertionError** – If the two matrices are not equal.

**Return type**

bool

### 3.6.3 bandhic.compute\_bin\_bias

`bandhic.compute_bin_bias(hic_coo, verbose=False, bias_lowerbound=0.5, bias_upperbound=2)`

Compute bias values for Hi-C contact matrices using the Knight-Ruiz normalization algorithm.

**Parameters**

- **hic\_coo** (`scipy.sparse.coo_array`) – A sparse COO matrix representing Hi-C contact data.
- **verbose** (`bool`, *optional*) – If True, print detailed information during processing. Default is False.

**Returns**

- **bias** (`numpy.ndarray`) – A 1D array containing the bias values for each bin in the Hi-C matrix.
- **is\_valid** (`bool`) – A boolean indicating whether the bias vector is valid (mean and median within typical range).

#### Examples

```
>>> import scipy.sparse as sps
>>> from bandhic import compute_bin_bias
>>> # Create a sample sparse COO matrix
>>> row = np.array([0, 1, 2, 0, 1, 2])
>>> col = np.array([0, 1, 2, 1, 2, 0])
>>> data = np.array([10, 10, 10, 0, 0, 0])
>>> hic_coo = sps.coo_array((data, (row, col)), shape=(3, 3))
>>> bias, is_valid = compute_bin_bias(hic_coo, verbose=False)
>>> print(bias)
[1. 1. 1.]
>>> print(is_valid)
True
```

---

## Notes

This function removes a specified percentage of the most sparse bins from the Hi-C matrix before computing the bias values. The Knight-Ruiz normalization algorithm is applied to the modified matrix to compute the bias. The function iteratively removes bins with low interaction counts until a valid bias vector is obtained. The bias vector is expected to have a mean and median close to 1, indicating balanced interaction frequencies across bins.

### 3.6.4 bandhic.call\_tad

`bandhic.call_tad(hic_matrix, resolution, chrom_short, window_size_bp=200000, min_TAD_size=None, stat_filter=True, verbose=False)`

Detect TADs from a Hi-C matrix using the TopDom algorithm. Different from the *TopDom* function, this function accepts a Hi-C matrix in either *band\_hic\_matrix* format or as a dense numpy array. This function is designed to be used with the BandHiC package and provides a convenient interface for TAD detection.

#### Parameters

- **hic\_matrix** (*band\_hic\_matrix* or *np.ndarray*) – The Hi-C matrix, either as a *band\_hic\_matrix* object or a dense numpy array.
- **resolution** (*int*) – The resolution of the Hi-C matrix.
- **chrom\_short** (*str*) – The chromosome name without ‘chr’ prefix.
- **window\_size\_bp** (*int*, *optional*) – The size of the window to consider for detecting TADs, in base pairs. Default is 200000.
- **min\_TAD\_size** (*int*, *optional*) – The minimum size of a TAD to be considered valid, in base pairs. Default is None.
- **stat\_filter** (*bool*, *optional*) – Whether to apply statistical filtering to remove false positives. Default is True.
- **verbose** (*bool*, *optional*) – Whether to print detailed information during processing. Default is False.

#### Returns

- **domains** (*pd.DataFrame*) – DataFrame containing detected TADs with columns ‘chr’, ‘from.id’, ‘from.coord’, ‘to.id’, ‘to.coord’, ‘tag’.
- **bins** (*pd.DataFrame*) – DataFrame containing bin information columns ‘id’, ‘chr’, ‘from.coord’, ‘to.coord’, ‘local.ext’, ‘mean.cf’, ‘pvalue’.

### 3.6.5 bandhic.topdom

`bandhic.topdom(hic_matrix, bins, window_size, stat_filter=True, verbose=False)`

Detect TADs using TopDom algorithm.

#### Parameters

- **hic\_matrix** (*band\_hic\_matrix* or *np.ndarray*) – The Hi-C matrix, either as a *band\_hic\_matrix* object or a dense numpy array.
- **bins** (*pd.DataFrame*) – DataFrame containing bin information with columns ‘chr’, ‘from.coord’, ‘to.coord’.
- **window\_size** (*int*) – Size of the window to consider for detecting TADs.

- **stat\_filter** (*bool, optional*) – Whether to apply statistical filtering to remove false positives. Default is True.
- **verbose** (*bool, optional*) – Whether to print detailed information during processing. Default is False.

#### Returns

- **domains** (*pd.DataFrame*) – DataFrame containing detected TADs with columns 'chr', 'from.id', 'from.coord', 'to.id', 'to.coord', 'tag'.
- **bins** (*pd.DataFrame*) – Updated DataFrame containing bin information with additional columns 'local.ext', 'mean.cf', 'pvalue'.

### 3.6.6 bandhic.apa

`bandhic.apa(hic_path, resolution, loops_df, window=10, region_width=6, min_peak_dist=0.0, max_peak_dist=8000000, njobs=-1)`

Compute Aggregate Peak Analysis (APA) around a set of loop anchors.

This is a lightweight Python port of the Juicer/juicebox APA workflow. For each loop (x, y), we extract a (2\*window+1)×(2\*window+1) cutout centered at (x, y), accumulate cutouts across loops, and report common APA normalizations and region-based summary statistics.

#### Notes

- Loops are processed per chromosome. For each chromosome, a contact map is loaded from `hic_path` at the requested `resolution` using `bandhic.straw_chr` (with `normalization='KR'` in the current implementation).
- Loop distance filtering is applied in **bin units** after converting loop coordinates to bins. By default, `max_peak_dist` is interpreted as a genomic distance in bp and converted to bins via `resolution`.

#### Parameters

- **hic\_path** (*str*) – Path to an input .hic file.
- **resolution** (*int*) – Hi-C bin size in base pairs.
- **loops\_df** (*DataFrame*) – Loop list as a DataFrame (e.g., BEDPE). The current implementation expects at least the columns '#chr1', 'chr2', 'x1', and 'y1' (coordinates in bp). Only intra-chromosomal loops (`#chr1 == chr2`) are used.
- **window** (*int*) – Number of bins to include on each side of the loop center; the final cutout size is 2\*window+1.
- **region\_width** (*int*) – Corner box size (in bins) used for APA region statistics.
- **min\_peak\_dist** (*float*) – Minimum loop distance from the diagonal (in bins, after binning).
- **max\_peak\_dist** (*float*) – Maximum loop distance from the diagonal (in bp; converted to bins as `max_peak_dist // resolution` for filtering and matrix loading).
- **njobs** (*Optional[int]*) – Number of parallel worker processes for loop cutout extraction. -1 uses up to `os.cpu_count()` workers (capped by the number of loops).

---

### Returns

Aggregated APA matrices (raw and normalized), enhancement scores, peak counts, and region-based summary statistics.

### Return type

APAResult

## 3.6.7 bandhic.hiccups

`bandhic.hiccups(hic_file, chroms, resolutions, fdr=None, peak_width=None, window=None, thresholds=(0.02, 1.5, 1.75, 2.0), centroid_radius=None, max_loop_dist_bp=2000000, kr_neighborhood=5, matrix_size=512, norm='KR', out_path=None, n_jobs=-1)`

Run the HiCCUPS loop-calling algorithm on Hi-C data using the BandHiC data structure.

This function provides a high-level interface to a BandHiC-based reimplementation of the HiCCUPS algorithm. It supports single- or multi-resolution loop detection across one or more chromosomes and internally applies multiprocessing to accelerate computation on large, high-resolution Hi-C matrices.

### Parameters

- **hic\_file** (*str*) – Path to the input Hi-C file. Supported formats include `.hic`, `.cool`, and `.mcool`.
- **chroms** (*str or list of str*) – Chromosome name or list of chromosome names (e.g., `"chr1"` or `["chr1", "chr2"]`).
- **resolutions** (*list of int*) – List of bin resolutions (in base pairs) at which HiCCUPS will be executed.
- **fdr** (*list of float or float, optional*) – Target false discovery rate (FDR) for loop detection at each resolution. If a single value is provided, it is applied to all resolutions.
- **peak\_width** (*list of int or int, optional*) – Peak half-width (in bins) for each resolution. Defaults follow Juicer recommendations if not specified.
- **window** (*list of int or int, optional*) – Donut window size (in bins) for each resolution. Defaults follow Juicer recommendations if not specified.
- **thresholds** (*list of float, optional*) – Threshold parameters [`fdrsum`, `oe1`, `oe2`, `oe3`] used for HiCCUPS filtering and cross-resolution merging.
- **centroid\_radius** (*list of int or int, optional*) – Radius (in base pairs) for clustering nearby loop pixels into centroids at each resolution.
- **max\_loop\_dist\_bp** (*int, optional*) – Maximum genomic distance (in base pairs) for loop search. Default is 2,000,000 bp.
- **kr\_neighborhood** (*int, optional*) – Radius (in bins) used to mask low-quality bins based on the KR normalization vector.
- **matrix\_size** (*int, optional*) – Reserved parameter for compatibility; not used in the current implementation.
- **norm** (*str, optional*) – Normalization method to use (e.g., `"KR"`, `"VC"`). Default is `"KR"`.
- **out\_path** (*str, optional*) – Path to write the output loops in BEDPE format. If `None`, results are not written to disk.
- **n\_jobs** (*int, optional*) – Number of parallel worker processes to use. `-1` uses all available CPUs.

---

**Returns**

**loops** – List of detected chromatin loops after multi-resolution merging.

**Return type**

list of Feature2D

### 3.6.8 bandhic.load\_norm\_vector\_from\_hic

`bandhic.load_norm_vector_from_hic(hic_file, chrom, resolution, norm)`

Load a normalization vector from a .hic file for a given chromosome and resolution.

**Parameters**

- **hic\_file** (*str*) – Path to the .hic file.
- **chrom** (*str*) – Chromosome name, e.g., 'chr1' or '1'.
- **resolution** (*int*) – Bin size in base pairs.
- **norm** (*str*) – Normalization method, e.g., 'KR', 'VC', etc.

**Returns**

Normalization vector for the specified chromosome and resolution.

**Return type**

np.ndarray

**Raises**

**RuntimeError** – If the normalization vector cannot be found for the specified parameters.

### 3.6.9 bandhic.load\_norm\_vector\_from\_cooler

`bandhic.load_norm_vector_from_cooler(cool_path, chrom, resolution, norm)`

Load a normalization vector from a .cool or .mcool file for a given chromosome and resolution.

**Parameters**

- **cool\_path** (*str*) – Path to the .cool or .mcool file.
- **chrom** (*str*) – Chromosome name, e.g., 'chr1'.
- **resolution** (*int*) – Bin size in base pairs.
- **norm** (*str*) – Normalization method, e.g., 'KR', 'VC', etc.

**Returns**

Normalization vector for the specified chromosome and resolution.

**Return type**

np.ndarray

**Raises**

**RuntimeError** – If the normalization vector cannot be found for the specified parameters.

---

### 3.6.10 bandhic.scc

`bandhic.scc(A, B, max_k=None, min_n=10, method='pearson')`

Stratified correlation coefficient (SCC) for Hi-C using BandHiC.

#### Parameters

- **A** (`band_hic_matrix`) – Two BandHiC matrices (same shape, same `diag_num`)
- **B** (`band_hic_matrix`) – Two BandHiC matrices (same shape, same `diag_num`)
- **max\_k** (`int`) – Max diagonal (distance bin) to consider
- **min\_n** (`int`) – Minimum number of valid pixels per stratum
- **method** (`{ "pearson", "spearman" }`) –

#### Returns

- **scc** (`float`)
- **per\_diag** (`dict {k: rho_k}`)

#### Examples

```
>>> import bandhic as bh
>>> A = bh.straw_chr("./data/GSE130275_mESC_WT_combined_1.3B_microc.hic", "chr1",
↪ 100_000, diag_num=20)
>>> B = A.copy()
>>> scc, per_diag = scc(A, B)
>>> scc
1.0
```



## INDICES AND TABLES

- `genindex`
- `modindex`
- `search`



## Symbols

`__array__()` (*bandhic.band\_hic\_matrix* method), 97  
`__array_priority__()` (*bandhic.band\_hic\_matrix* method), 98  
`__bool__()` (*bandhic.band\_hic\_matrix* method), 100  
`__getitem__()` (*bandhic.band\_hic\_matrix* method), 35  
`__hash__()` (*bandhic.band\_hic\_matrix* method), 99  
`__init__()` (*bandhic.band\_hic\_matrix* method), 25  
`__iter__()` (*bandhic.band\_hic\_matrix* method), 42  
`__len__()` (*bandhic.band\_hic\_matrix* method), 99  
`__repr__()` (*bandhic.band\_hic\_matrix* method), 97  
`__setitem__()` (*bandhic.band\_hic\_matrix* method), 37  
`__str__()` (*bandhic.band\_hic\_matrix* method), 97

## A

`absolute()` (*bandhic.band\_hic\_matrix* method), 55  
`absolute()` (*in module bandhic*), 111  
`add()` (*bandhic.band\_hic\_matrix* method), 56  
`add()` (*in module bandhic*), 112  
`add_mask()` (*bandhic.band\_hic\_matrix* method), 28  
`add_mask_row_col()` (*bandhic.band\_hic\_matrix* method), 30  
`all()` (*bandhic.band\_hic\_matrix* method), 52  
`all()` (*in module bandhic*), 153  
`any()` (*bandhic.band\_hic\_matrix* method), 52  
`any()` (*in module bandhic*), 153  
`apa()` (*in module bandhic*), 157  
`arccos()` (*bandhic.band\_hic\_matrix* method), 56  
`arccos()` (*in module bandhic*), 112  
`arccosh()` (*bandhic.band\_hic\_matrix* method), 57  
`arccosh()` (*in module bandhic*), 113  
`arcsin()` (*bandhic.band\_hic\_matrix* method), 57  
`arcsin()` (*in module bandhic*), 113  
`arcsinh()` (*bandhic.band\_hic\_matrix* method), 58  
`arcsinh()` (*in module bandhic*), 114  
`arctan()` (*bandhic.band\_hic\_matrix* method), 58  
`arctan()` (*in module bandhic*), 115  
`arctan2()` (*bandhic.band\_hic\_matrix* method), 59  
`arctan2()` (*in module bandhic*), 115  
`arctanh()` (*bandhic.band\_hic\_matrix* method), 60  
`arctanh()` (*in module bandhic*), 116

`assert_band_matrix_equal()` (*in module bandhic*), 155  
`astype()` (*bandhic.band\_hic\_matrix* method), 96

## B

`band_hic_matrix` (*class in bandhic*), 24  
`bin_num` (*bandhic.band\_hic\_matrix* attribute), 24  
`bitwise_and()` (*bandhic.band\_hic\_matrix* method), 60  
`bitwise_and()` (*in module bandhic*), 116  
`bitwise_or()` (*bandhic.band\_hic\_matrix* method), 61  
`bitwise_or()` (*in module bandhic*), 117  
`bitwise_xor()` (*bandhic.band\_hic\_matrix* method), 61  
`bitwise_xor()` (*in module bandhic*), 117

## C

`call_tad()` (*in module bandhic*), 156  
`cbrt()` (*bandhic.band\_hic\_matrix* method), 62  
`cbrt()` (*in module bandhic*), 118  
`clear_mask()` (*bandhic.band\_hic\_matrix* method), 31  
`clip()` (*bandhic.band\_hic\_matrix* method), 45  
`clip()` (*in module bandhic*), 154  
`compute_bin_bias()` (*in module bandhic*), 155  
`conj()` (*bandhic.band\_hic\_matrix* method), 62  
`conj()` (*in module bandhic*), 119  
`conjugate()` (*bandhic.band\_hic\_matrix* method), 63  
`conjugate()` (*in module bandhic*), 119  
`cooler_all_chrs()` (*in module bandhic*), 109  
`cooler_chr()` (*in module bandhic*), 108  
`copy()` (*bandhic.band\_hic\_matrix* method), 96  
`cos()` (*bandhic.band\_hic\_matrix* method), 63  
`cos()` (*in module bandhic*), 120  
`cosh()` (*bandhic.band\_hic\_matrix* method), 64  
`cosh()` (*in module bandhic*), 120  
`count_in_band()` (*bandhic.band\_hic\_matrix* method), 33  
`count_in_band_masked()` (*bandhic.band\_hic\_matrix* method), 32  
`count_masked()` (*bandhic.band\_hic\_matrix* method), 32  
`count_out_band()` (*bandhic.band\_hic\_matrix* method), 34

---

`count_out_band_masked()` (*bandhic.band\_hic\_matrix method*), 33  
`count_unmasked()` (*bandhic.band\_hic\_matrix method*), 32

## D

`data` (*bandhic.band\_hic\_matrix attribute*), 24  
`default_value` (*bandhic.band\_hic\_matrix attribute*), 25  
`deg2rad()` (*bandhic.band\_hic\_matrix method*), 64  
`deg2rad()` (*in module bandhic*), 121  
`degrees()` (*bandhic.band\_hic\_matrix method*), 65  
`degrees()` (*in module bandhic*), 121  
`diag()` (*bandhic.band\_hic\_matrix method*), 40  
`diag_num` (*bandhic.band\_hic\_matrix attribute*), 24  
`divide()` (*bandhic.band\_hic\_matrix method*), 65  
`divide()` (*in module bandhic*), 122  
`divmod()` (*bandhic.band\_hic\_matrix method*), 66  
`divmod()` (*in module bandhic*), 122  
`drop_mask()` (*bandhic.band\_hic\_matrix method*), 29  
`drop_mask_row_col()` (*bandhic.band\_hic\_matrix method*), 31  
`dtype` (*bandhic.band\_hic\_matrix attribute*), 24  
`dump()` (*bandhic.band\_hic\_matrix method*), 100

## E

`equal()` (*bandhic.band\_hic\_matrix method*), 67  
`equal()` (*in module bandhic*), 123  
`exp()` (*bandhic.band\_hic\_matrix method*), 67  
`exp()` (*in module bandhic*), 124  
`exp2()` (*bandhic.band\_hic\_matrix method*), 68  
`exp2()` (*in module bandhic*), 124  
`expm1()` (*bandhic.band\_hic\_matrix method*), 68  
`expm1()` (*in module bandhic*), 125  
`extract_row()` (*bandhic.band\_hic\_matrix method*), 41  
`eye()` (*in module bandhic*), 102  
`eye_like()` (*in module bandhic*), 104

## F

`fabs()` (*bandhic.band\_hic\_matrix method*), 69  
`fabs()` (*in module bandhic*), 125  
`filled()` (*bandhic.band\_hic\_matrix method*), 44  
`float_power()` (*bandhic.band\_hic\_matrix method*), 69  
`float_power()` (*in module bandhic*), 126  
`floor_divide()` (*bandhic.band\_hic\_matrix method*), 70  
`floor_divide()` (*in module bandhic*), 126  
`fmod()` (*bandhic.band\_hic\_matrix method*), 70  
`fmod()` (*in module bandhic*), 127  
`full()` (*in module bandhic*), 102  
`full_like()` (*in module bandhic*), 104

## G

`gcd()` (*bandhic.band\_hic\_matrix method*), 71

`gcd()` (*in module bandhic*), 128  
`get_mask()` (*bandhic.band\_hic\_matrix method*), 28  
`get_mask_row_col()` (*bandhic.band\_hic\_matrix method*), 31  
`get_values()` (*bandhic.band\_hic\_matrix method*), 39  
`greater()` (*bandhic.band\_hic\_matrix method*), 72  
`greater()` (*in module bandhic*), 128  
`greater_equal()` (*bandhic.band\_hic\_matrix method*), 72  
`greater_equal()` (*in module bandhic*), 129

## H

`heaviside()` (*bandhic.band\_hic\_matrix method*), 73  
`heaviside()` (*in module bandhic*), 129  
`hiccups()` (*in module bandhic*), 158  
`hypot()` (*bandhic.band\_hic\_matrix method*), 73  
`hypot()` (*in module bandhic*), 130

## I

`init_mask()` (*bandhic.band\_hic\_matrix method*), 27  
`invert()` (*bandhic.band\_hic\_matrix method*), 74  
`invert()` (*in module bandhic*), 131  
`itercols()` (*bandhic.band\_hic\_matrix method*), 43  
`iterrows()` (*bandhic.band\_hic\_matrix method*), 43  
`iterwindows()` (*bandhic.band\_hic\_matrix method*), 42

## L

`lcm()` (*bandhic.band\_hic\_matrix method*), 74  
`lcm()` (*in module bandhic*), 131  
`left_shift()` (*bandhic.band\_hic\_matrix method*), 75  
`left_shift()` (*in module bandhic*), 132  
`less()` (*bandhic.band\_hic\_matrix method*), 76  
`less()` (*in module bandhic*), 132  
`less_equal()` (*bandhic.band\_hic\_matrix method*), 76  
`less_equal()` (*in module bandhic*), 133  
`load_norm_vector_from_cooler()` (*in module bandhic*), 159  
`load_norm_vector_from_hic()` (*in module bandhic*), 159  
`load_npz()` (*in module bandhic*), 106  
`log()` (*bandhic.band\_hic\_matrix method*), 77  
`log()` (*in module bandhic*), 134  
`log10()` (*bandhic.band\_hic\_matrix method*), 78  
`log10()` (*in module bandhic*), 135  
`log1p()` (*bandhic.band\_hic\_matrix method*), 77  
`log1p()` (*in module bandhic*), 134  
`log2()` (*bandhic.band\_hic\_matrix method*), 78  
`log2()` (*in module bandhic*), 135  
`logaddexp()` (*bandhic.band\_hic\_matrix method*), 79  
`logaddexp()` (*in module bandhic*), 136  
`logaddexp2()` (*bandhic.band\_hic\_matrix method*), 79  
`logaddexp2()` (*in module bandhic*), 136  
`logical_and()` (*bandhic.band\_hic\_matrix method*), 80  
`logical_and()` (*in module bandhic*), 137

---

`logical_or()` (*bandhic.band\_hic\_matrix method*), 80  
`logical_or()` (*in module bandhic*), 138  
`logical_xor()` (*bandhic.band\_hic\_matrix method*), 81  
`logical_xor()` (*in module bandhic*), 138

## M

`mask` (*bandhic.band\_hic\_matrix attribute*), 25  
`mask_row_col` (*bandhic.band\_hic\_matrix attribute*), 25  
`matrix_equal()` (*in module bandhic*), 154  
`max()` (*bandhic.band\_hic\_matrix method*), 47  
`max()` (*in module bandhic*), 152  
`maximum()` (*bandhic.band\_hic\_matrix method*), 82  
`maximum()` (*in module bandhic*), 139  
`mean()` (*bandhic.band\_hic\_matrix method*), 48  
`mean()` (*in module bandhic*), 152  
`memory_usage()` (*bandhic.band\_hic\_matrix method*), 98  
`min()` (*bandhic.band\_hic\_matrix method*), 46  
`min()` (*in module bandhic*), 152  
`minimum()` (*bandhic.band\_hic\_matrix method*), 82  
`minimum()` (*in module bandhic*), 139  
`mod()` (*bandhic.band\_hic\_matrix method*), 83  
`mod()` (*in module bandhic*), 140  
`multiply()` (*bandhic.band\_hic\_matrix method*), 83  
`multiply()` (*in module bandhic*), 141

## N

`negative()` (*bandhic.band\_hic\_matrix method*), 84  
`negative()` (*in module bandhic*), 141  
`normalize()` (*bandhic.band\_hic\_matrix method*), 53  
`normalize()` (*in module bandhic*), 154  
`not_equal()` (*bandhic.band\_hic\_matrix method*), 84  
`not_equal()` (*in module bandhic*), 142

## O

`ones()` (*in module bandhic*), 101  
`ones_like()` (*in module bandhic*), 103

## P

`positive()` (*bandhic.band\_hic\_matrix method*), 85  
`positive()` (*in module bandhic*), 142  
`power()` (*bandhic.band\_hic\_matrix method*), 86  
`power()` (*in module bandhic*), 143  
`prod()` (*bandhic.band\_hic\_matrix method*), 50  
`prod()` (*in module bandhic*), 153  
`ptp()` (*bandhic.band\_hic\_matrix method*), 51  
`ptp()` (*in module bandhic*), 153

## R

`rad2deg()` (*bandhic.band\_hic\_matrix method*), 86  
`rad2deg()` (*in module bandhic*), 143  
`radians()` (*bandhic.band\_hic\_matrix method*), 87  
`radians()` (*in module bandhic*), 144

`reciprocal()` (*bandhic.band\_hic\_matrix method*), 87  
`reciprocal()` (*in module bandhic*), 144  
`remainder()` (*bandhic.band\_hic\_matrix method*), 88  
`remainder()` (*in module bandhic*), 145  
`right_shift()` (*bandhic.band\_hic\_matrix method*), 88  
`right_shift()` (*in module bandhic*), 146  
`rint()` (*bandhic.band\_hic\_matrix method*), 89  
`rint()` (*in module bandhic*), 146

## S

`save_npz()` (*in module bandhic*), 105  
`scc()` (*in module bandhic*), 160  
`set_diag()` (*bandhic.band\_hic\_matrix method*), 41  
`set_values()` (*bandhic.band\_hic\_matrix method*), 39  
`shape` (*bandhic.band\_hic\_matrix attribute*), 24  
`sign()` (*bandhic.band\_hic\_matrix method*), 89  
`sign()` (*in module bandhic*), 147  
`sin()` (*bandhic.band\_hic\_matrix method*), 90  
`sin()` (*in module bandhic*), 147  
`sinh()` (*bandhic.band\_hic\_matrix method*), 90  
`sinh()` (*in module bandhic*), 148  
`sqrt()` (*bandhic.band\_hic\_matrix method*), 91  
`sqrt()` (*in module bandhic*), 148  
`square()` (*bandhic.band\_hic\_matrix method*), 91  
`square()` (*in module bandhic*), 149  
`std()` (*bandhic.band\_hic\_matrix method*), 49  
`std()` (*in module bandhic*), 153  
`straw_all_chrs()` (*in module bandhic*), 107  
`straw_chr()` (*in module bandhic*), 106  
`subtract()` (*bandhic.band\_hic\_matrix method*), 92  
`subtract()` (*in module bandhic*), 149  
`sum()` (*bandhic.band\_hic\_matrix method*), 48  
`sum()` (*in module bandhic*), 152

## T

`tan()` (*bandhic.band\_hic\_matrix method*), 92  
`tan()` (*in module bandhic*), 150  
`tanh()` (*bandhic.band\_hic\_matrix method*), 93  
`tanh()` (*in module bandhic*), 150  
`tocoo()` (*bandhic.band\_hic\_matrix method*), 95  
`tocsr()` (*bandhic.band\_hic\_matrix method*), 95  
`todense()` (*bandhic.band\_hic\_matrix method*), 94  
`topdom()` (*in module bandhic*), 156  
`true_divide()` (*bandhic.band\_hic\_matrix method*), 93  
`true_divide()` (*in module bandhic*), 151

## U

`unmask()` (*bandhic.band\_hic\_matrix method*), 29

## V

`var()` (*bandhic.band\_hic\_matrix method*), 50  
`var()` (*in module bandhic*), 153

---

## Z

`zeros()` (*in module bandhic*), [101](#)

`zeros_like()` (*in module bandhic*), [103](#)